

# | Equation of Motion Time Discretization

| Newton's 2nd Law as a  
first-order system

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{v} \\ \mathbf{M}\dot{\mathbf{v}} = \mathbf{f}(\mathbf{x}, \mathbf{v}) \end{cases}$$

BDF1

$$\dot{\mathbf{y}}_t \approx \frac{\mathbf{y}_{t+1} - \mathbf{y}_t}{h}$$

| Explicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h \mathbf{v}^- \\ \mathbf{v}^+ = \mathbf{v}^- + h \mathbf{M}^{-1} \mathbf{f}(\mathbf{x}^-, \mathbf{v}^-) \end{cases}$$

| Implicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h \mathbf{v}^+ \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h \mathbf{f}(\mathbf{x}^+, \mathbf{v}^+) \end{cases}$$

## | Implicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h \mathbf{v}^+ \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h \mathbf{f}(\mathbf{x}^+, \mathbf{v}^+) \end{cases}$$



$$\mathbf{r}(\mathbf{x}^+) = \frac{1}{h^2} \mathbf{M}(\mathbf{x}^+ - \tilde{\mathbf{x}}) - \mathbf{f}(\mathbf{x}^+) = \mathbf{0}$$



**Non-Linear  
System of  
Equations**

$$\frac{1}{h^2} \begin{bmatrix} m_1 & & & \\ & m_2 & & \\ & & \ddots & \\ & & & m_n \end{bmatrix} \begin{bmatrix} \mathbf{x}_1^+ - \tilde{\mathbf{x}}_1 \\ \mathbf{x}_2^+ - \tilde{\mathbf{x}}_2 \\ \vdots \\ \mathbf{x}_n^+ - \tilde{\mathbf{x}}_n \end{bmatrix} - \begin{bmatrix} \mathbf{f}_1(\mathbf{x}^+) \\ \mathbf{f}_2(\mathbf{x}^+) \\ \vdots \\ \mathbf{f}_n(\mathbf{x}^+) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

# | Implicit Euler

$$\mathbf{r}(\mathbf{x}^+) = \frac{1}{h^2} \mathbf{M} (\mathbf{x}^+ - \tilde{\mathbf{x}}) - \mathbf{f}(\mathbf{x}^+) = \mathbf{0}$$

Unknown

$\mathbf{x}^- + h \mathbf{v}^-$

Force: Non-Linear Function

Timestep Length

Diagonal Mass Matrix:  $\begin{bmatrix} m_1 & & \\ & \ddots & \\ & & m_n \end{bmatrix}$

The diagram illustrates the Implicit Euler method equation:  $\mathbf{r}(\mathbf{x}^+) = \frac{1}{h^2} \mathbf{M} (\mathbf{x}^+ - \tilde{\mathbf{x}}) - \mathbf{f}(\mathbf{x}^+) = \mathbf{0}$ . Annotations include: 'Unknown' pointing to  $\mathbf{x}^+$ ;  $\mathbf{x}^- + h \mathbf{v}^-$  pointing to  $\tilde{\mathbf{x}}$ ; 'Force: Non-Linear Function' pointing to  $\mathbf{f}(\mathbf{x}^+)$ ; 'Timestep Length' pointing to  $h$ ; and 'Diagonal Mass Matrix' pointing to  $\mathbf{M}$ , which is represented as a diagonal matrix with elements  $m_1, \dots, m_n$ .

# | Newton-Raphson Method

We have:

$$\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^n$$

We want:

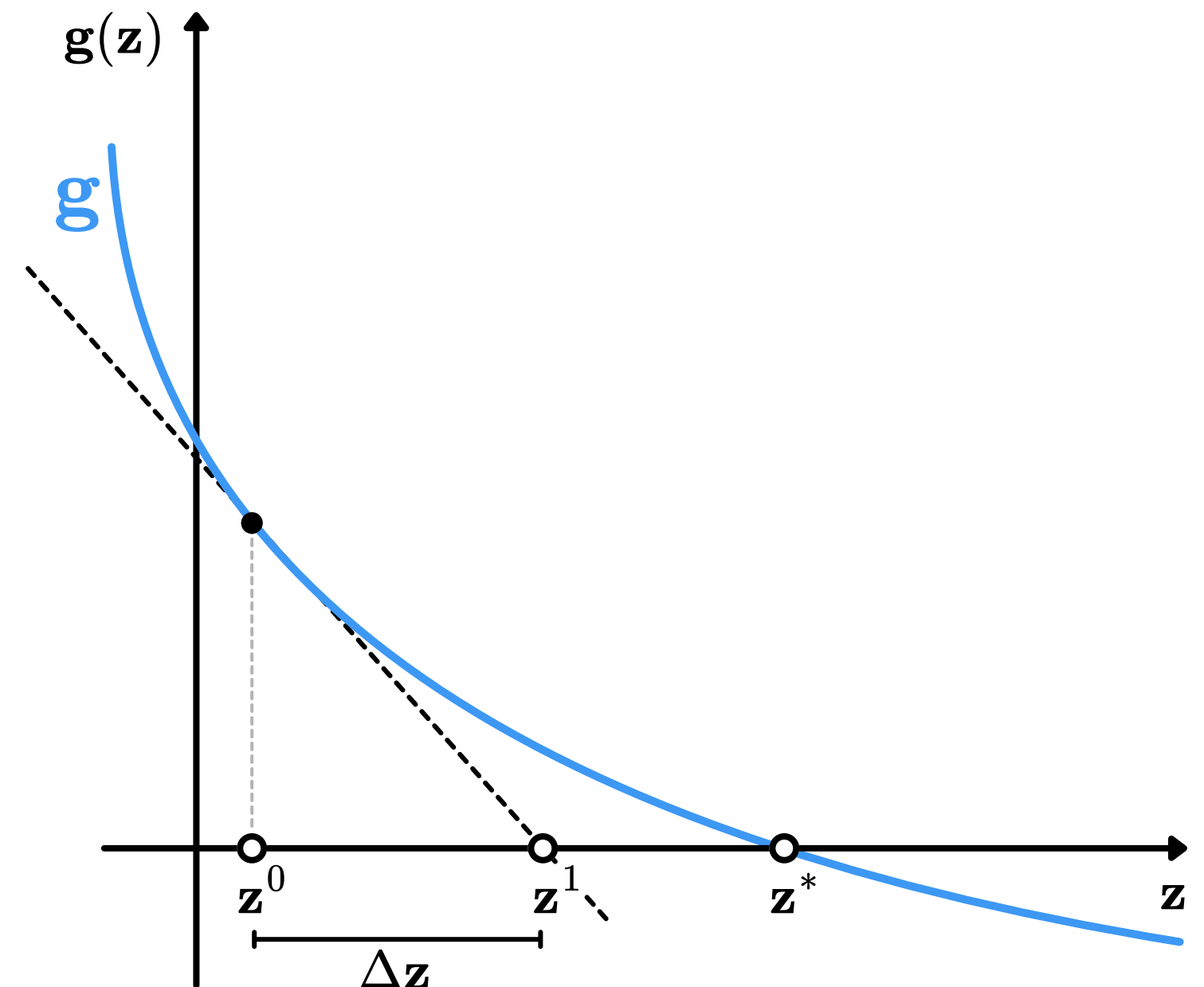
$$\mathbf{g}(\mathbf{z}^*) = \mathbf{0}$$

Linearize around a guess:

$$\mathbf{g}(\mathbf{z}^{(k)} + \Delta \mathbf{z}) \approx \mathbf{g}(\mathbf{z}^{(k)}) + \frac{\partial \mathbf{g}}{\partial \mathbf{z}}(\mathbf{z}^{(k)}) \Delta \mathbf{z} = \mathbf{0}$$

Linear System of Equations

$$\frac{\partial \mathbf{g}}{\partial \mathbf{z}} \Delta \mathbf{z} = -\mathbf{g}(\mathbf{z}^{(k)})$$
$$\mathbf{z}^{(k+1)} = \mathbf{z}^{(k)} + \Delta \mathbf{z}$$



# | Newton-Raphson Method for Implicit Dynamic System

We have:

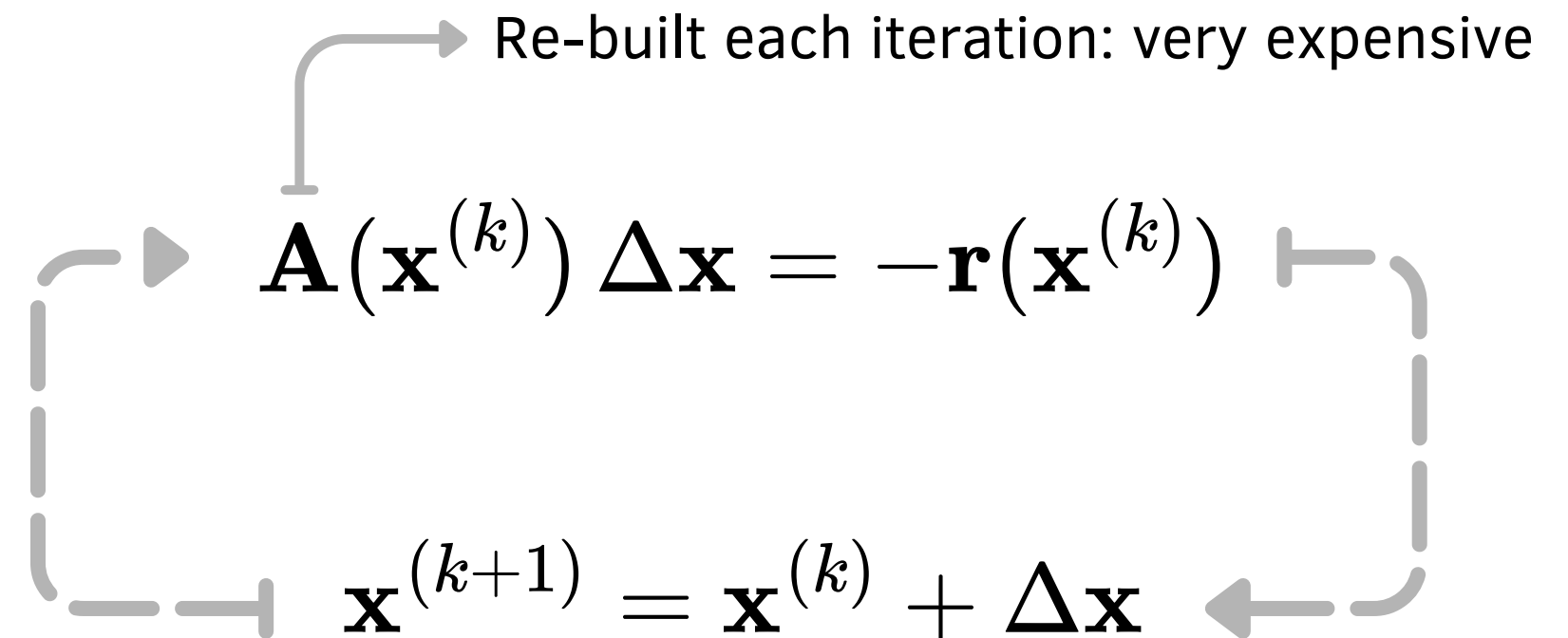
$$\mathbf{r} : \mathbb{R}^{3n} \rightarrow \mathbb{R}^{3n} \quad \text{such that:} \quad \mathbf{r}(\mathbf{x}) = \frac{1}{h^2} \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) - \mathbf{f}(\mathbf{x})$$

We want:

$$\mathbf{r}(\mathbf{x}^+) = \mathbf{0}$$

Linearize around a guess:

$$\frac{\partial \mathbf{r}}{\partial \mathbf{x}} = \underbrace{\frac{1}{h^2} \mathbf{M} - \frac{\partial \mathbf{f}}{\partial \mathbf{x}}}_{\mathbf{A} \in \mathbb{R}^{3n \times 3n}}$$



# | Newton-Raphson Method Single Iteration

Expanded:

$$\underbrace{\left[ \mathbf{M} - h^2 \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right]}_{\mathbf{A}(\mathbf{x}^-)} \Delta \mathbf{x} = \underbrace{h^2 \mathbf{f}(\mathbf{x}^-) + h \mathbf{M} \mathbf{v}^-}_{\mathbf{b}(\mathbf{x}^-)}$$

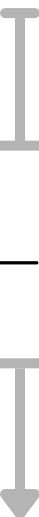
Algorithm:

- 1 **build:**  $\mathbf{A}, \mathbf{b}$
- 2 **solve:**  $\mathbf{A} \Delta \mathbf{x} = \mathbf{b}$
- 3  $\mathbf{x}^+ = \mathbf{x}^- + \Delta \mathbf{x}$
- 4  $\mathbf{v}^+ = \frac{1}{h} (\mathbf{x}^+ - \mathbf{x}^-)$

# | Recasting Root-Finding as Minimization

Does exist a:  $g(\mathbf{x})$  such that:  $\nabla g(\mathbf{x}) = \mathbf{r}(\mathbf{x})$

$$g(\mathbf{x}) = \frac{1}{2h^2} (\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + U(\mathbf{x})$$

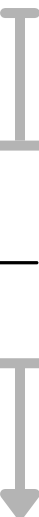

$$\mathbf{f}(\mathbf{x}) = -\frac{\partial U}{\partial \mathbf{x}}(\mathbf{x}) \quad \text{Conservative Forces}$$

$$\nabla g(\mathbf{x}) = \frac{1}{h^2} \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) - \mathbf{f}(\mathbf{x}) = \mathbf{r}(\mathbf{x})$$

# | Recasting Root-Finding as Minimization

Does exists a:  $g(\mathbf{x})$  such that:  $\nabla g(\mathbf{x}) = \mathbf{r}(\mathbf{x})$

$$g(\mathbf{x}) = \frac{1}{2h^2} (\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M} (\mathbf{x} - \tilde{\mathbf{x}}) + U(\mathbf{x})$$


$$\mathbf{f}(\mathbf{x}) = -\frac{\partial U}{\partial \mathbf{x}}(\mathbf{x})$$

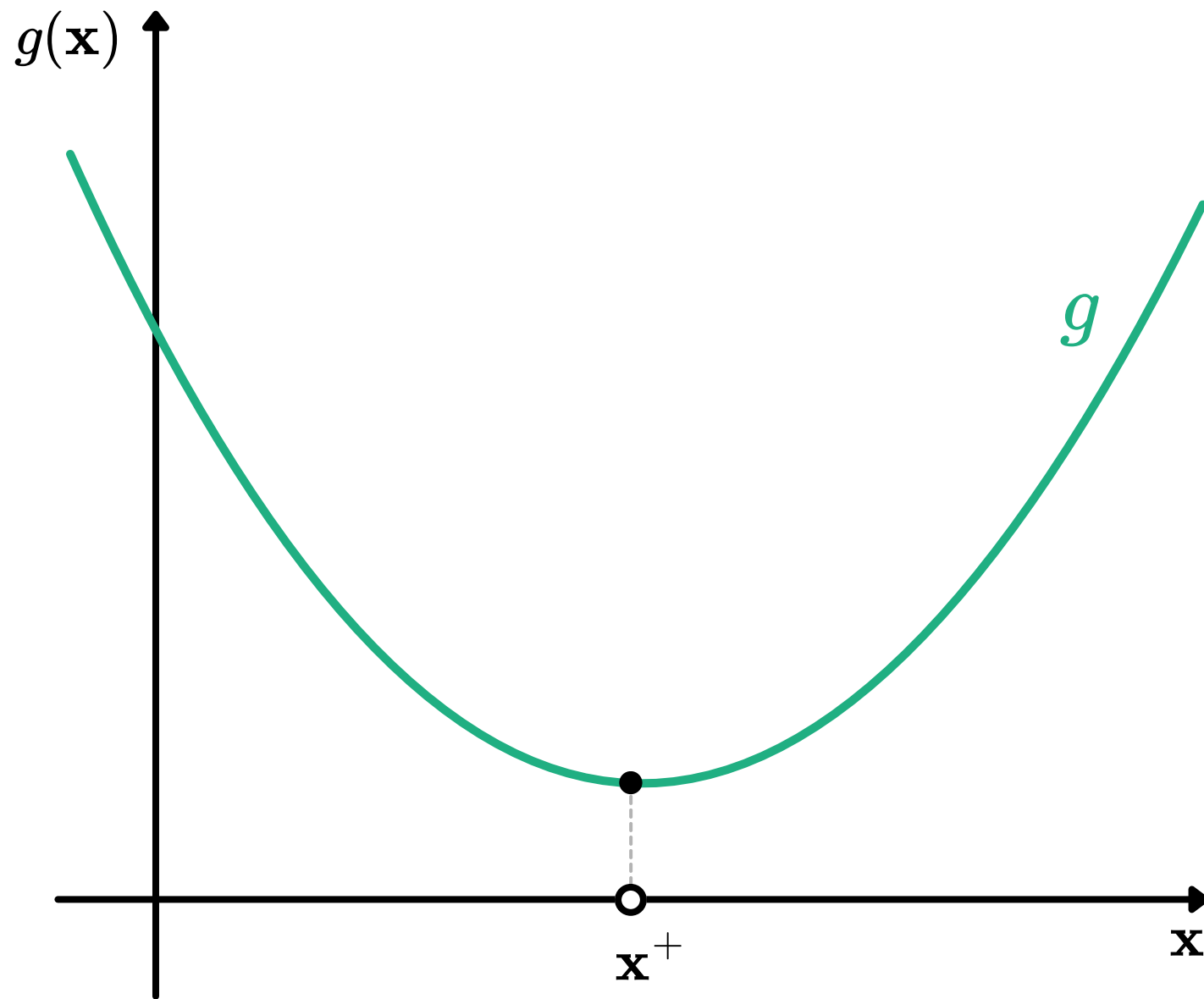
Conservative Forces

$$\mathbf{r}(\mathbf{x}) = \mathbf{0} \quad \Longleftrightarrow \quad \nabla g(\mathbf{x}) = \mathbf{0} \quad \begin{array}{c} \text{if Convex} \\ \Longleftrightarrow \end{array} \quad \min_{\mathbf{x}} g(\mathbf{x})$$

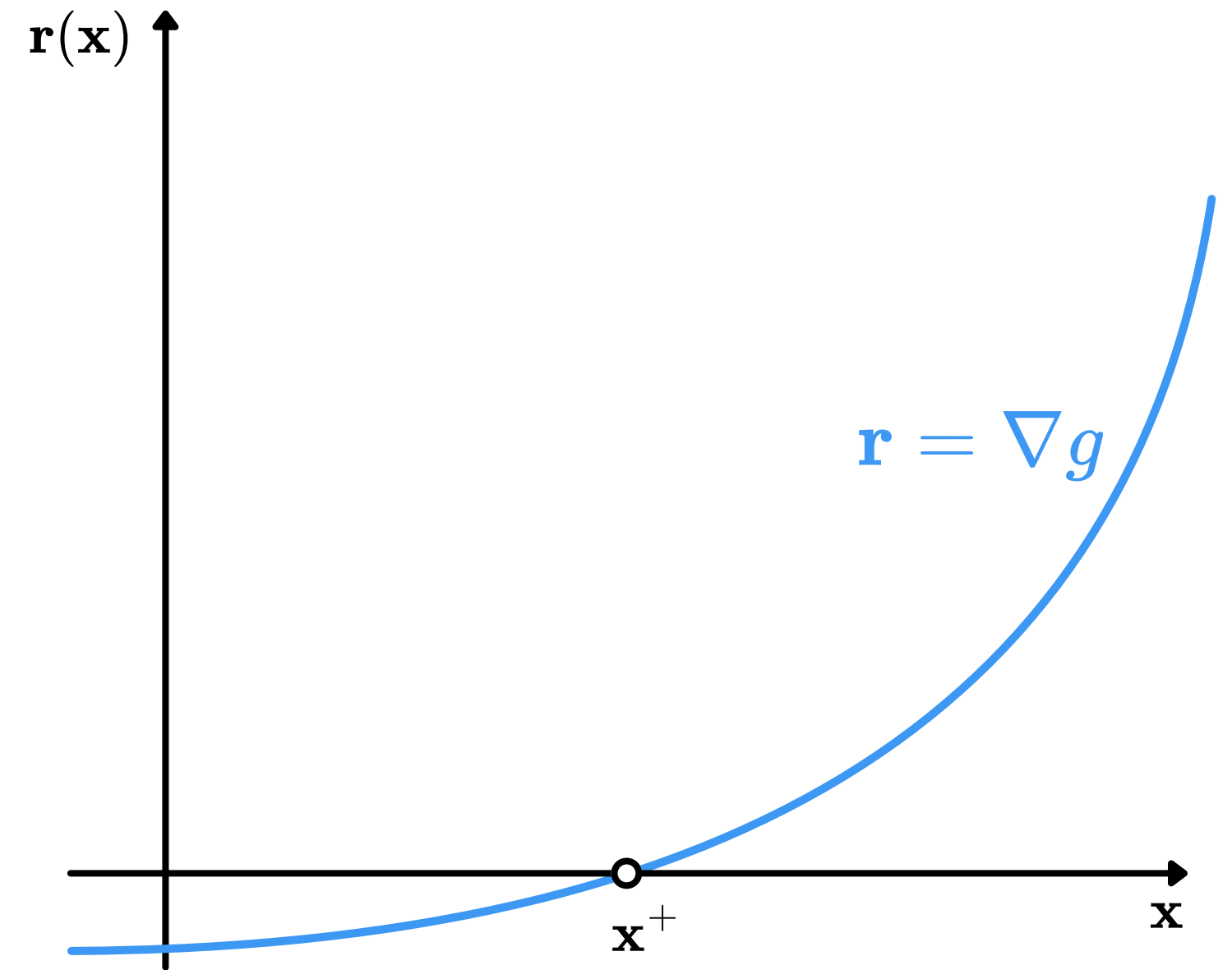
# | Root-Finding and Minimization Equivalence

Equivalent Problems:

$$\mathbf{x}^+ = \arg \min_{\mathbf{x}} g(\mathbf{x})$$



$$\mathbf{r}(\mathbf{x}^+) = 0$$



# | Descent Methods How to Solve the Minimization Problem

```
1 Initialize  $\mathbf{x}^{(0)}$ 
2 for  $k = 0, 1, \dots, N - 1$  do
3    $\Delta \mathbf{x} = \text{DescentDirection}()$ 
4    $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \cdot \Delta \mathbf{x}$ 
5  $\mathbf{x}^+ = \mathbf{x}^{(N)}$ 
6  $\mathbf{v}^+ = \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$ 
```

| Jacobian:

$$\mathbf{J} = \nabla g(\mathbf{x}) = \mathbf{r}(\mathbf{x}) \in \mathbb{R}^{3n}$$

| Hessian:

$$\mathbf{H} = \nabla^2 g(\mathbf{x}) = \mathbf{A}(\mathbf{x}) \in \mathbb{R}^{3n \times 3n}$$

---

| Gradient Descent:

$$\Delta \mathbf{x}^{(k)} = -\mathbf{J}^{(k)}$$

| Newton:

$$\Delta \mathbf{x}^{(k)} = -[\mathbf{H}^{(k)}]^{-1} \mathbf{J}^{(k)}$$

| Quasi-Newton:

$$\Delta \mathbf{x}^{(k)} = -\mathbf{P}^{(k)} \mathbf{J}^{(k)}, \quad \mathbf{P}^{(k)} \approx [\mathbf{H}^{(k)}]^{-1}$$

| Projective Dynamics:

$$\Delta \mathbf{x}^{(k)} = -\mathbf{L}_{\text{PD}}^{-1} \mathbf{J}_{\text{PD}}^{(k)}$$

$$\mathbf{J}_{\text{PD}} = \nabla g_{\text{PD}}(\mathbf{x}), \quad \mathbf{L}_{\text{PD}} \approx \mathbf{H} \text{ (fixed)}$$

| Baraff-Witkin:

$$\Delta \mathbf{x} = -\mathbf{H}^{-1} \mathbf{J}, \quad k = 1$$

# Newton's Method

# | Newton's Minimization Method = Newton-Raphson Root Finding's Method

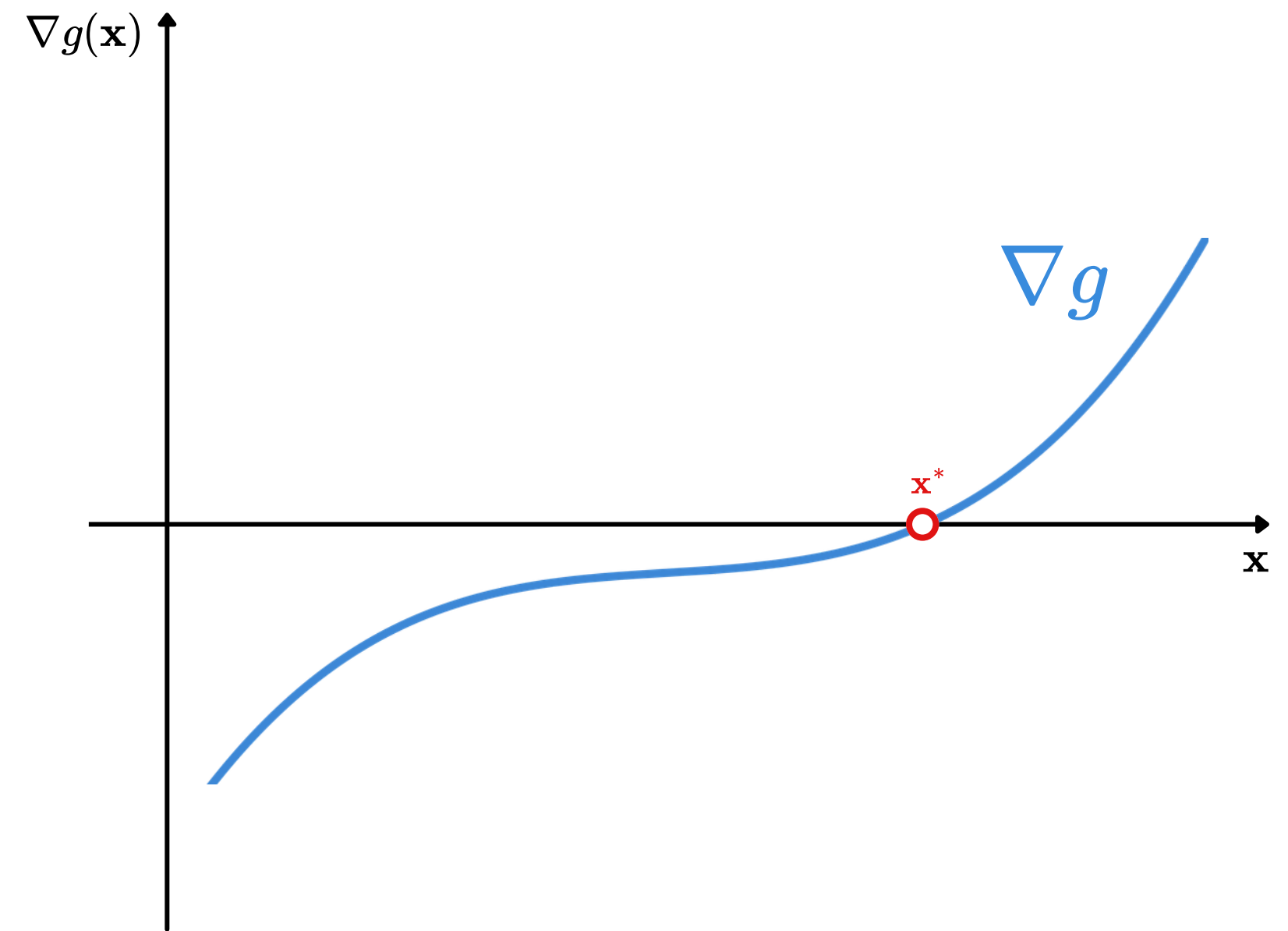
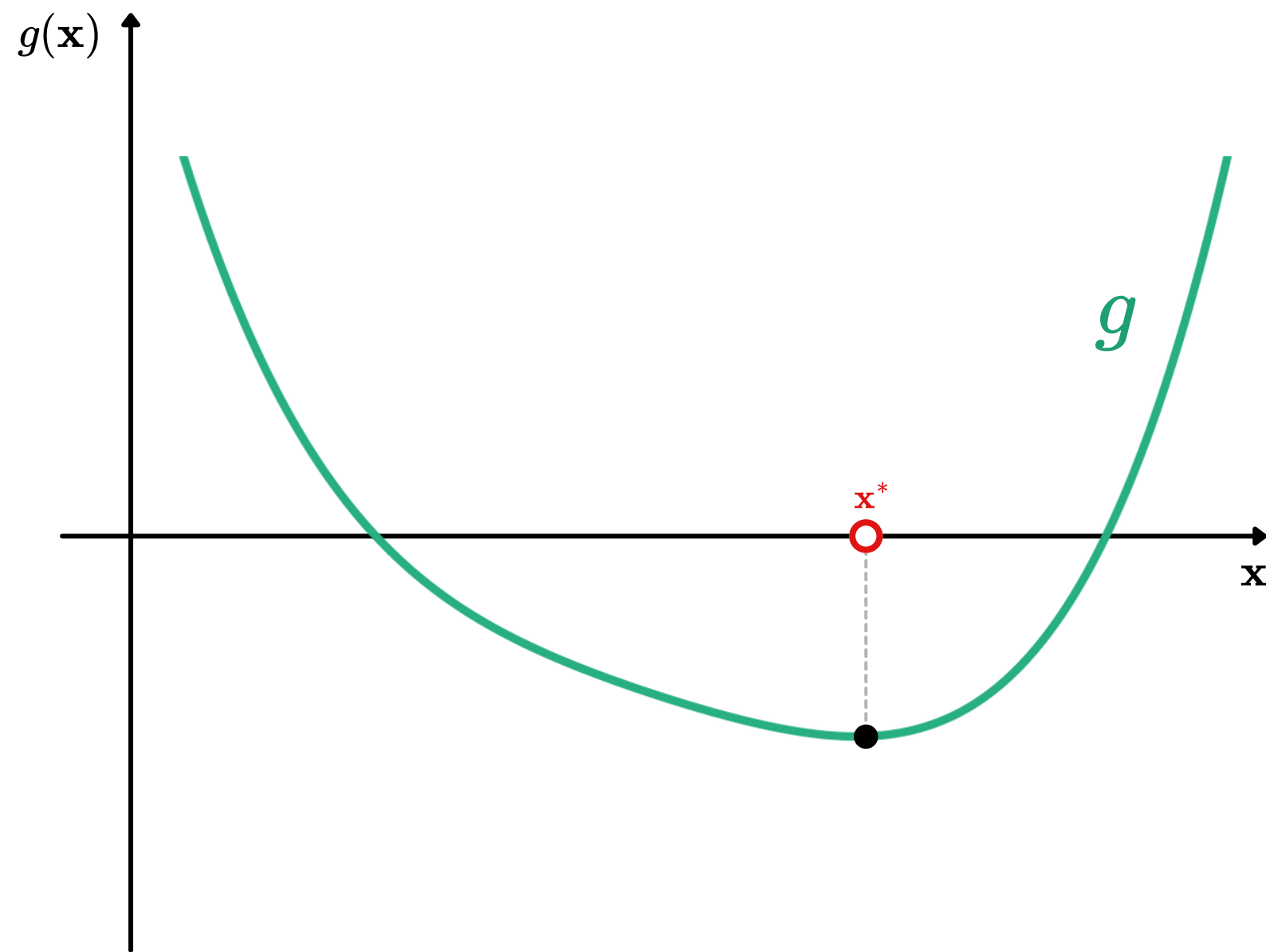
**function** NEWTONMETHOD( $\mathbf{x}^-$ ,  $\mathbf{v}^-$ ,  $N$ )

```
1 Initialize  $\mathbf{x}^{(0)}$ 
2 for  $k = 0, 1, \dots, N - 1$  do
3     BUILD  $\mathbf{J}^{(k)}$ ,  $\mathbf{H}^{(k)}$ 
4     SOLVE  $\mathbf{H}^{(k)} \Delta \mathbf{x} = -\mathbf{J}^{(k)}$ 
5      $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \Delta \mathbf{x}$ 
6  $\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$ 
7  $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$ 
```

**function** NEWTONRAPHSON( $\mathbf{x}^-$ ,  $\mathbf{v}^-$ ,  $N$ )

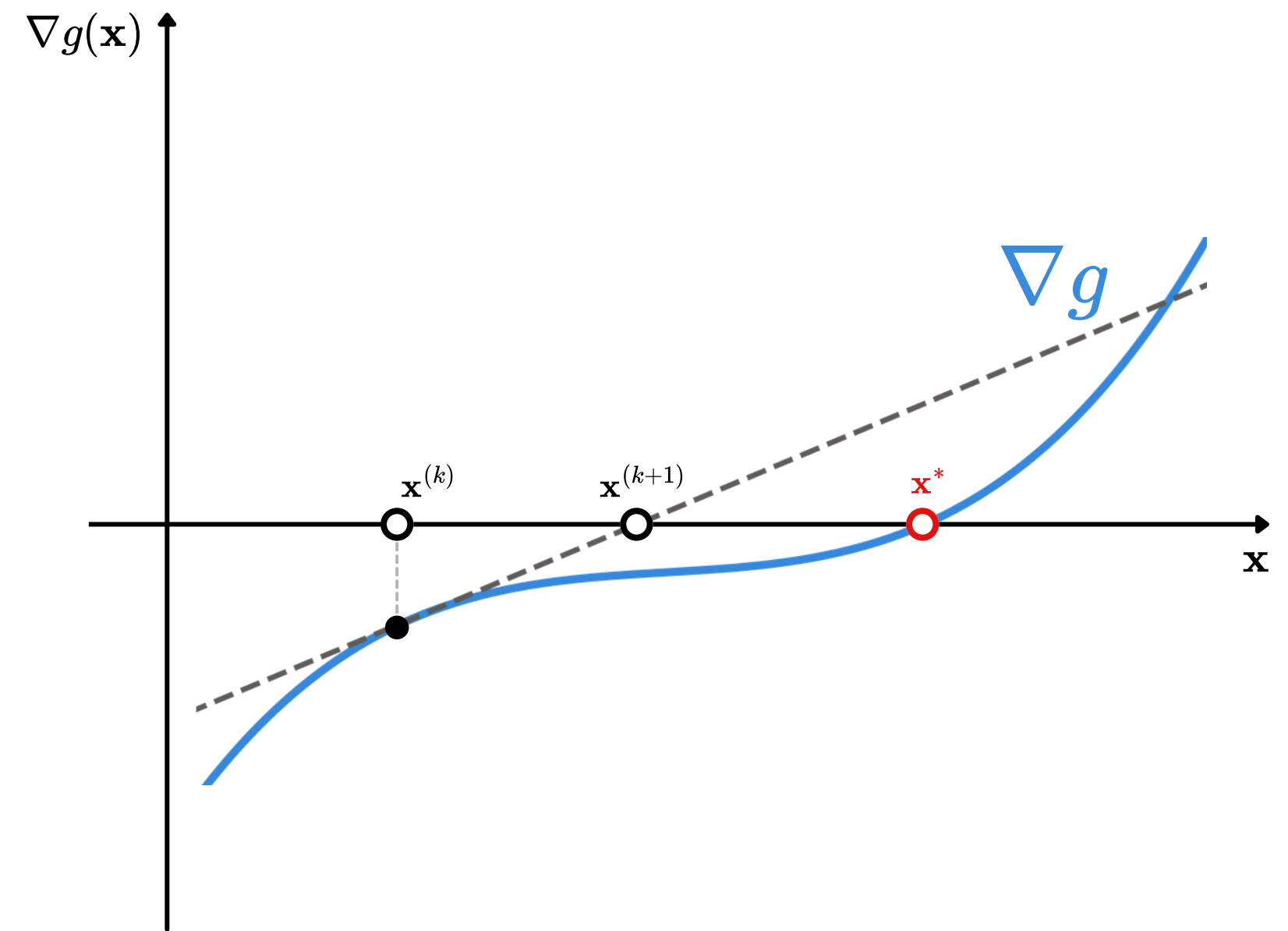
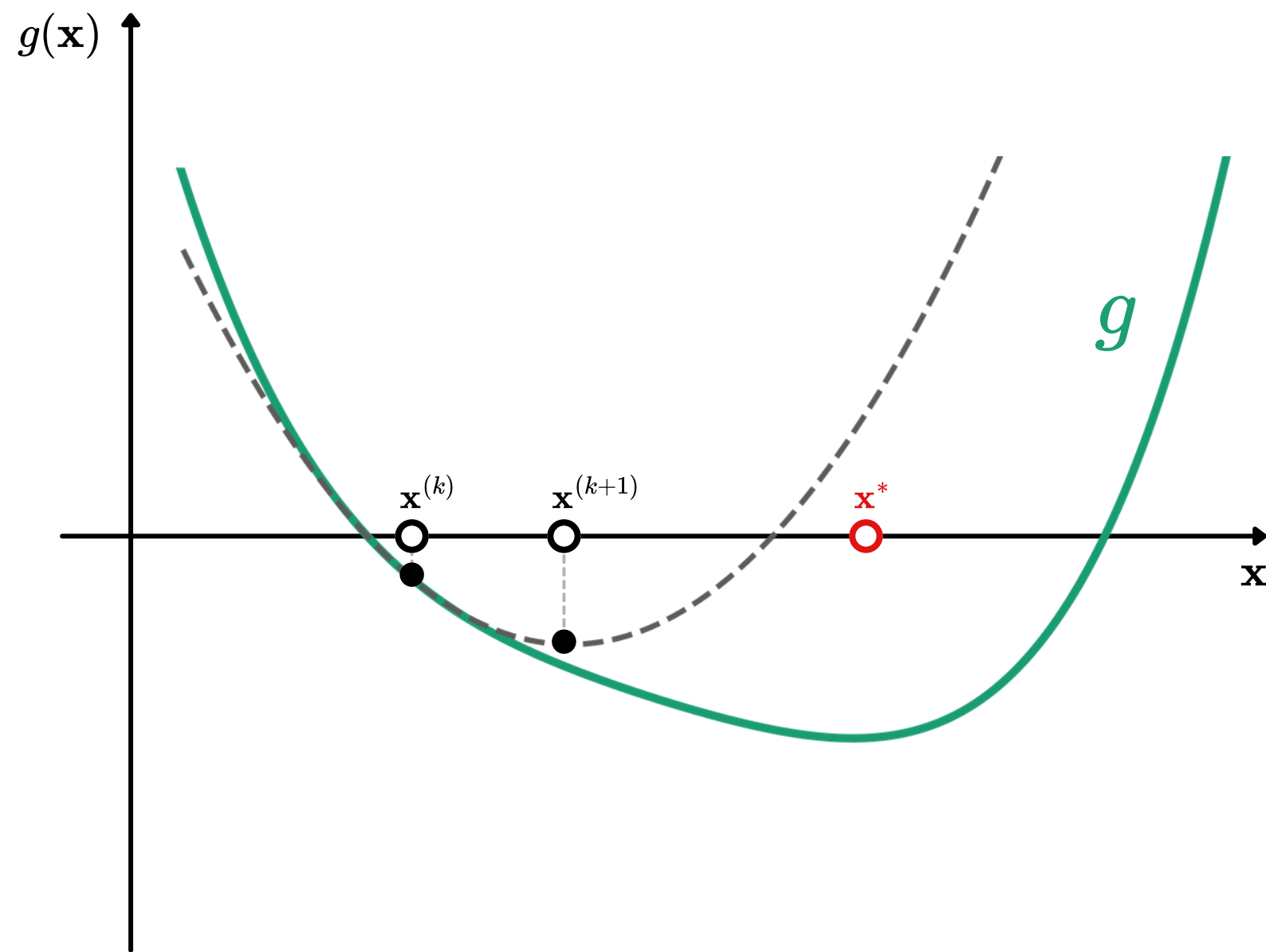
```
1 Initialize  $\mathbf{x}^{(0)}$ 
2 for  $k = 0, 1, \dots, N - 1$  do
3     BUILD  $\mathbf{A}^{(k)}$ ,  $\mathbf{b}^{(k)}$ 
4     SOLVE  $\mathbf{A}^{(k)} \Delta \mathbf{x} = \mathbf{b}^{(k)}$ 
5      $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \Delta \mathbf{x}$ 
6  $\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$ 
7  $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$ 
```

# | Convexity and Newton's Method When the Energy **IS** Convex



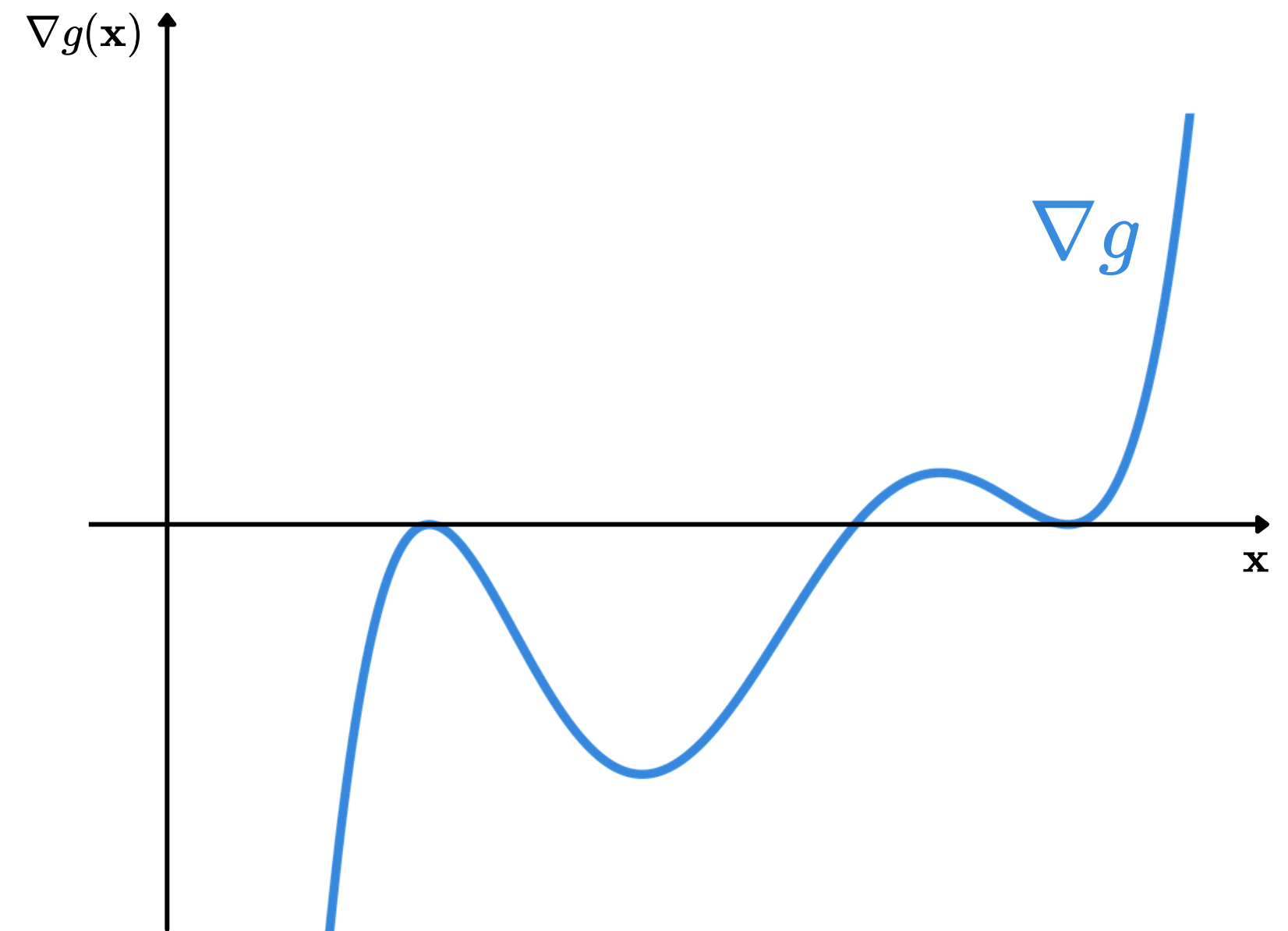
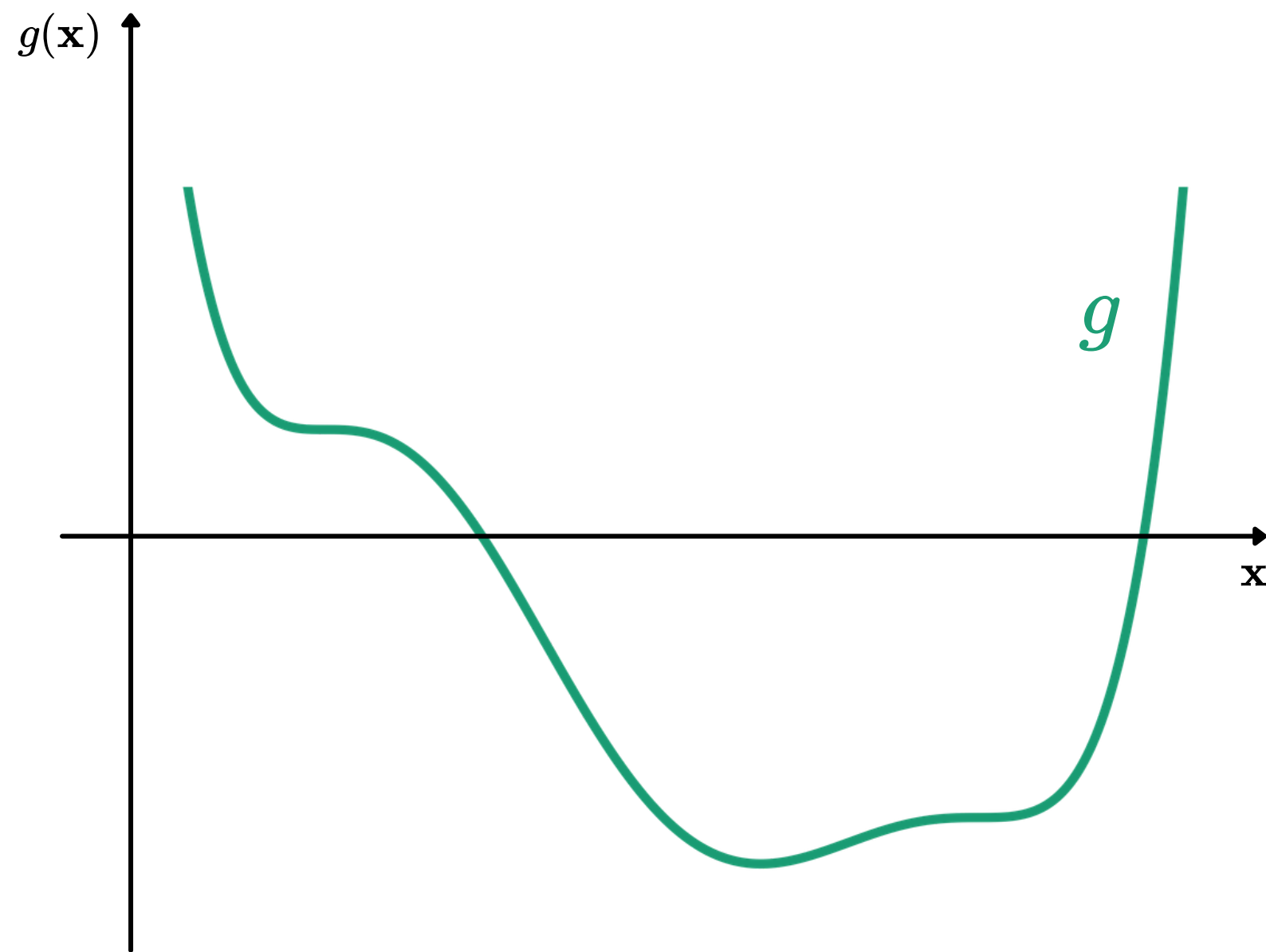
# | Convexity and Newton's Method When the Energy **IS** Convex

$$g(\mathbf{x}^{(k+1)}) < g(\mathbf{x}^{(k)})$$

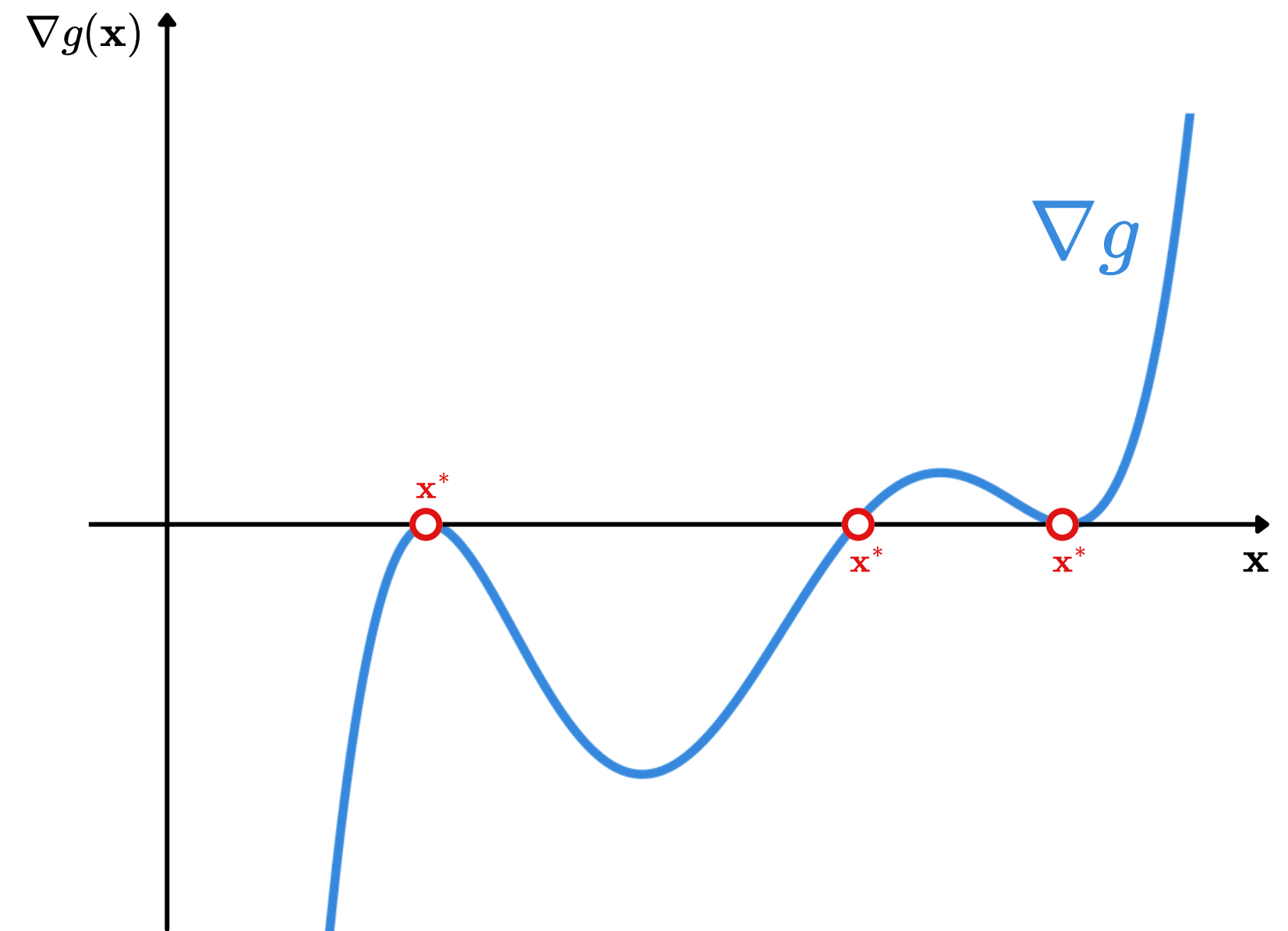
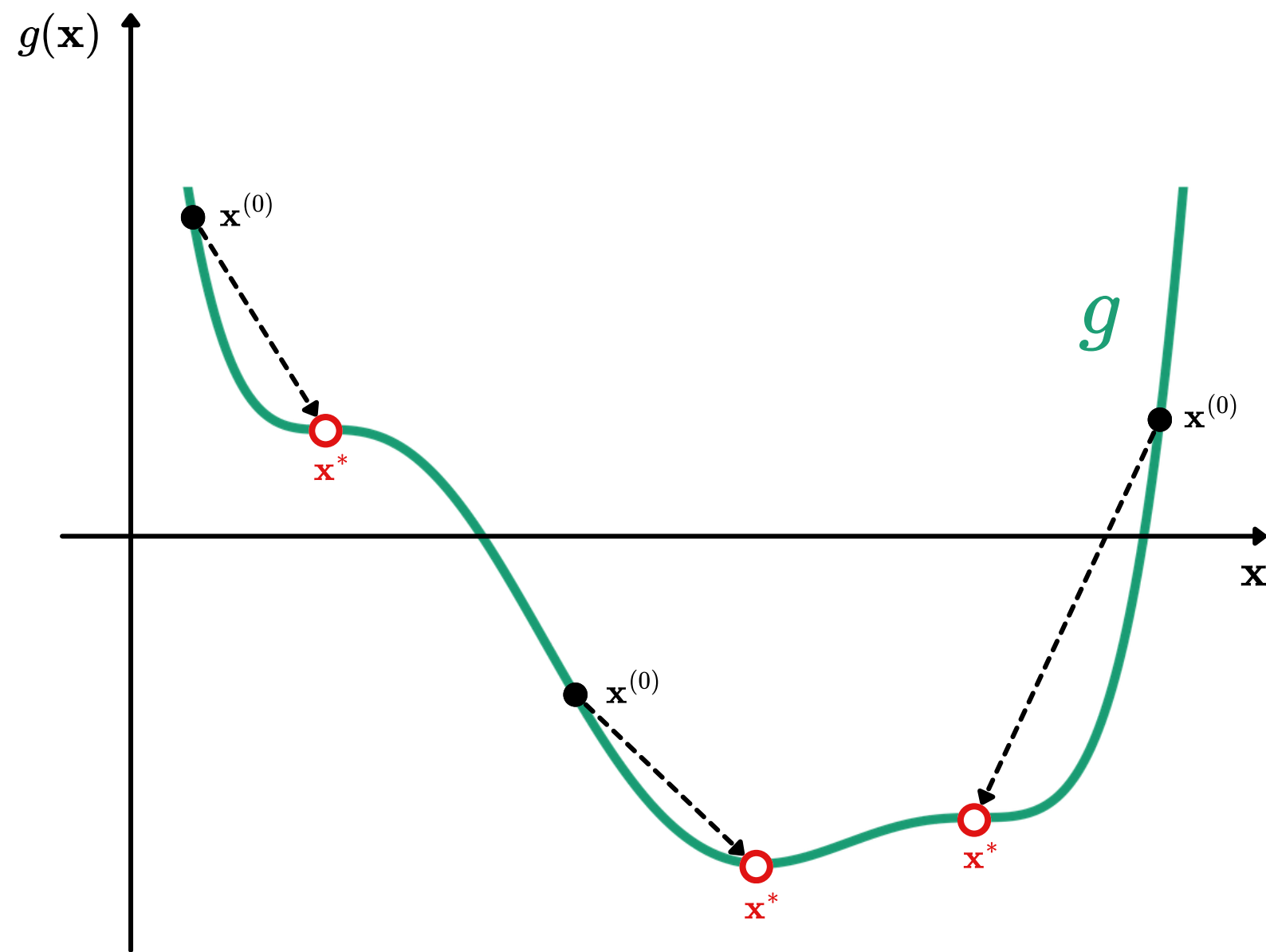


# | Convexity and Newton's Method When the Energy **IS NOT** Convex

- There might be multiple stationary points
- There might be areas where the curvature is negative



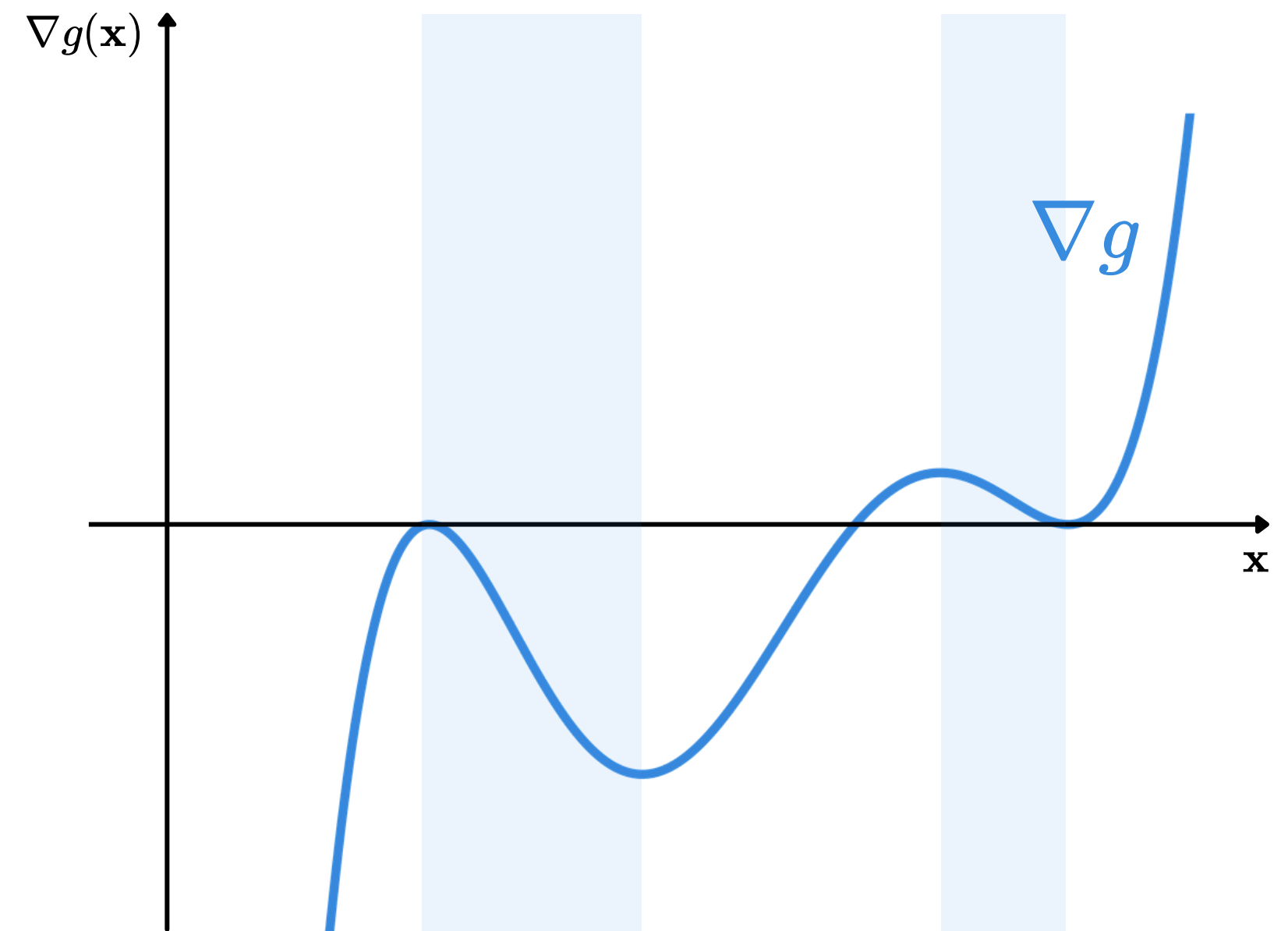
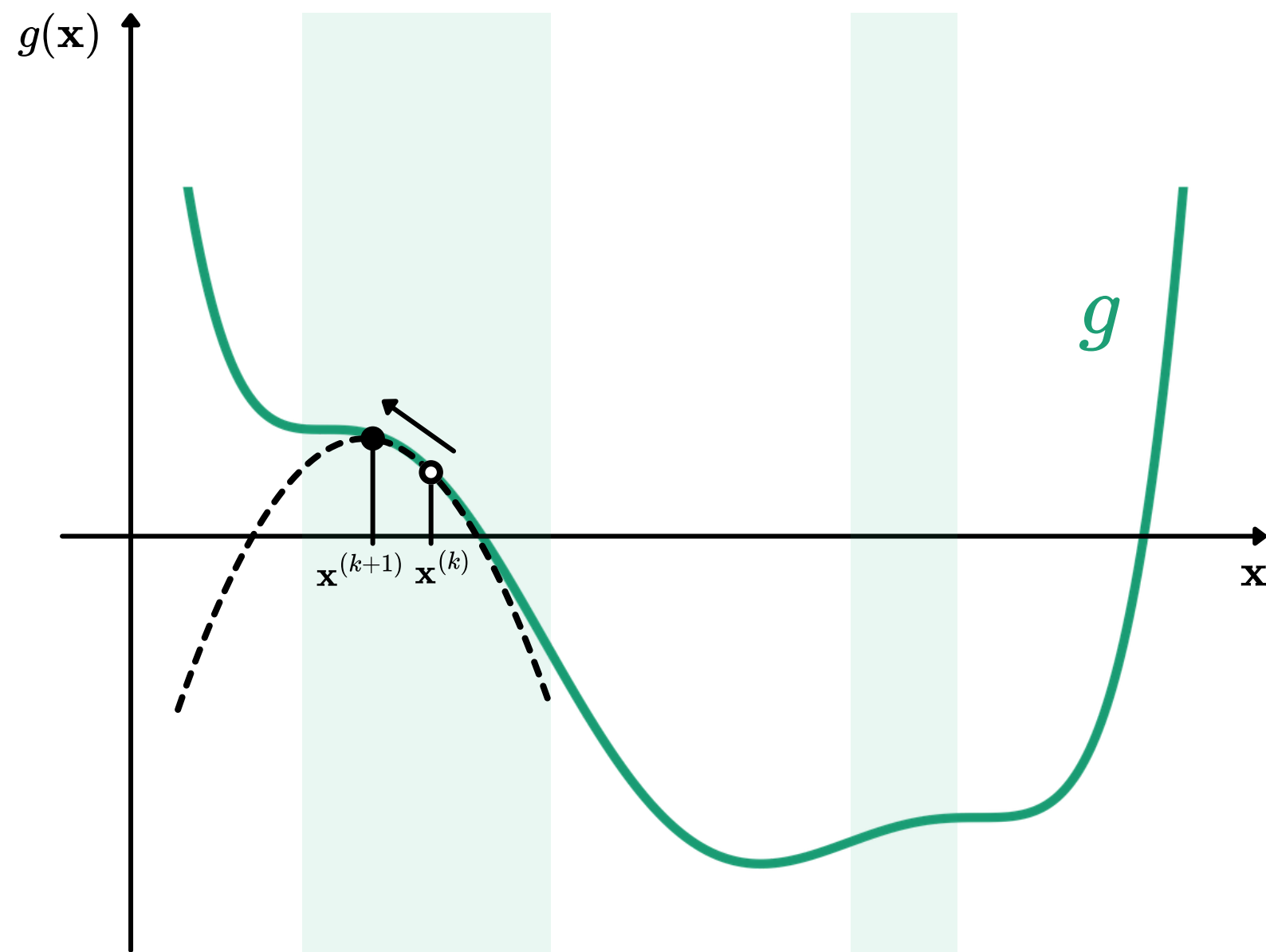
# | Convexity and Newton's Method: Multiple Stationary Points



# | Convexity and Newton's Method: Negative Curvature

If the curvature is negative, by taking the Newton's step, there might be an **increase** in Energy

$$g(\mathbf{x}^{(k+1)}) > g(\mathbf{x}^{(k)})$$



# | Convexity and Newton's Method: Solve the Negative Curvature Problem

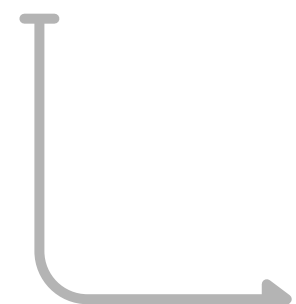
A matrix is SPD (**S**ymmetric **P**ositive **D**efinite), written  $\mathbf{A} \succ 0$  if it is symmetric and all its eigenvalues are positive:

$$\mathbf{A} = \mathbf{A}^\top \quad \text{and} \quad \mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top, \quad \lambda_i > 0 \quad \forall i.$$

- $\mathbf{A} \succ 0, \mathbf{B} \succ 0 \Rightarrow \mathbf{A} + \mathbf{B} \succ 0$
- $\mathbf{A} \succ 0, \mathbf{B} \not\succ 0 \Rightarrow \mathbf{A} + \mathbf{B}$  no guarantee

$\mathbf{H} \not\succ 0$  :  $\Delta \mathbf{x} = -\mathbf{H}^{-1} \mathbf{J}$  then  $\Delta \mathbf{x}$  may not be a descent direction

$\bar{\mathbf{H}} \succ 0$  :  $\Delta \mathbf{x} = -\bar{\mathbf{H}}^{-1} \mathbf{J}$  then  $\alpha \cdot \Delta \mathbf{x}$  is guaranteed to be a descent direction for a sufficiently small  $\alpha > 0$

 is the SPD-Projection of  $\mathbf{H}$

# Convexity and Newton's Method: Solve the Negative Curvature Problem

How to fix the Energy's Hessian

Always Convex

$$g(\mathbf{x}) = \frac{1}{2h^2} (\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M} (\mathbf{x} - \tilde{\mathbf{x}}) + U(\mathbf{x})$$

$\mathbf{H} = \frac{1}{h^2} \mathbf{M} + \nabla^2 U(\mathbf{x})$

Always SPD      Might not be SPD:  
we must **SPD-Project**

$\nabla^2 U(\mathbf{x}) \in \mathbb{R}^{3n \times 3n}$  : it is very expensive to SPD-Project such a big matrix

# Convexity and Newton's Method: Solve the Negative Curvature Problem

How is the Potential Energy Factor Constructed

$$U(\mathbf{x}) = \sum_e U_e(\mathbf{x}_e)$$

The total energy is the sum of single element energies. Each element (edge, triangle, tetrahedron) involves only a few particles (2, 3 or 4).

$$\nabla^2 U(\mathbf{x}) = \sum_e \nabla^2 U_e(\mathbf{x}_e)$$

$$\nabla^2 U_e(\mathbf{x}_e) \in \mathbb{R}^{6 \times 6 | 9 \times 9 | 12 \times 12}$$

So the Hessian is also a sum of small element Hessians. Each one can be easily SPD-projected independently and in parallel, then assembled into the full Hessian.

# Convexity and Newton's Method: Solve the Negative Curvature Problem

How an SPD-Projection is computed

How do we SPD-Project the single element energies:

$$\nabla^2 U_e(\mathbf{x}_e) \in \mathbb{R}^{6 \times 6 | 9 \times 9 | 12 \times 12}$$

Two ways:

- **Numerical:**

$$\nabla^2 U_e = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top \longrightarrow \bar{\lambda}_i = \max(\lambda_i, 0) \longrightarrow \overline{\nabla^2 U_e} = \mathbf{Q} \bar{\mathbf{\Lambda}} \mathbf{Q}^\top$$

- **Analytical:**

for many frequently used energies the eigenvalues and eigenvectors are known in closed form. This route is usually faster and more robust.

# | Analytical SPD Projection Quadratic Spring Potential

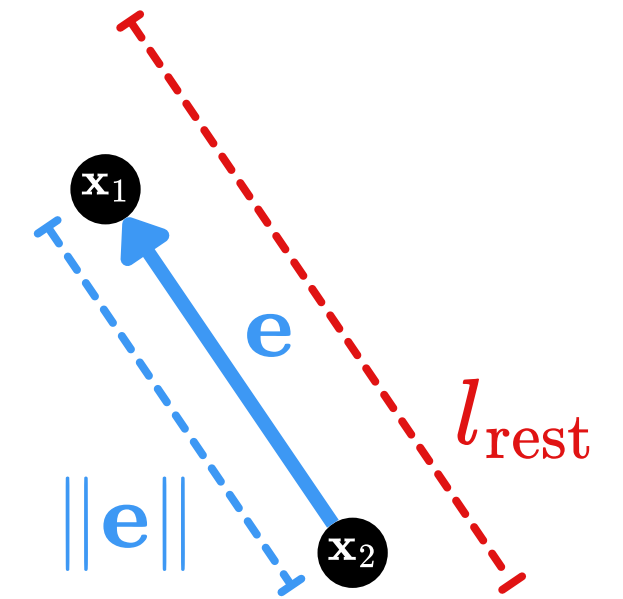
**Spring:**

$$\mathbf{x}_e = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{bmatrix} \in \mathbb{R}^6 \quad \mathbf{e} = \mathbf{x}_1 - \mathbf{x}_2$$

$$\hat{\mathbf{e}} = \frac{\mathbf{e}}{\|\mathbf{e}\|}$$

**Energy:**

$$U_e(\mathbf{x}_e) = \frac{k}{2} (\|\mathbf{e}\| - l_{\text{rest}})^2$$



**Hessian:**

$$\mathbf{H}_e = \begin{bmatrix} \mathbf{K} & -\mathbf{K} \\ -\mathbf{K} & \mathbf{K} \end{bmatrix} \in \mathbb{R}^{6 \times 6}$$

$$\mathbf{K} = k \left[ \hat{\mathbf{e}} \hat{\mathbf{e}}^\top + \left( 1 - \frac{l}{\|\mathbf{e}\|} \right) (\mathbf{I} - \hat{\mathbf{e}} \hat{\mathbf{e}}^\top) \right]$$

**Eigenvalues:**

$$\lambda_1 \quad \lambda_2 \quad \lambda_3 \quad \lambda_4 \quad \lambda_5 = \lambda_6$$

$$0, \quad 0, \quad 0, \quad 2k, \quad 2k \left( 1 - \frac{l}{\|\mathbf{e}\|} \right)$$

Can be negative

**SPD-Hessian:**

$$\bar{\mathbf{H}}_e = \begin{bmatrix} \bar{\mathbf{K}} & -\bar{\mathbf{K}} \\ -\bar{\mathbf{K}} & \bar{\mathbf{K}} \end{bmatrix} \in \mathbb{R}^{6 \times 6} \quad \bar{\mathbf{H}}_e \succeq 0$$

$$\bar{\mathbf{K}} = k \left[ \hat{\mathbf{e}} \hat{\mathbf{e}}^\top + \max \left( 1 - \frac{l}{\|\mathbf{e}\|}, 0 \right) (\mathbf{I} - \hat{\mathbf{e}} \hat{\mathbf{e}}^\top) \right]$$

## | SPD-Aware Newton's Method Algorithm

```
function SPDAWARENEWTONMETHOD( $\mathbf{x}^-$ ,  $\mathbf{v}^-$ )  
  2: Initialize  $\mathbf{x}^{(0)}$   
  3: for  $k = 0, 1, \dots, N - 1$  do  
    4:  $\mathbf{J} \leftarrow \nabla g(\mathbf{x}^{(k)})$   
    5:  $\bar{\mathbf{H}} \leftarrow \text{SPDPROJECTION}(\nabla^2 g(\mathbf{x}^{(k)}))$   
    6: SOLVE  $\bar{\mathbf{H}} \Delta \mathbf{x} = -\mathbf{J}$   
    7:  $\alpha \leftarrow \text{LINESEARCH}(\mathbf{x}^{(k)}, \Delta \mathbf{x}, \mathbf{J})$   
    8:  $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha \Delta \mathbf{x}$   
  
  9:  $\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$   
 10:  $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$   
end function
```

# | SPD-Aware Newton's Method → Projective Dynamics (PD)

At each iteration  $k$  we have to:

- $\bar{\mathbf{H}}^{(k)} = \text{SPDProjection}(\nabla^2 g(\mathbf{x}^{(k)}))$
- $\mathbf{C}^{(k)} = \text{Factorize}(\bar{\mathbf{H}}^{(k)})$
- $\Delta \mathbf{x} = \text{SolveLinearSystem}(\mathbf{C}^{(k)}, -\mathbf{J}^{(k)})$

## What is Factorization:

Factorization rewrites the matrix as a triangular matrix times its transpose:

$$\bar{\mathbf{H}} = \mathbf{C} \mathbf{C}^\top, \quad \mathbf{C} \text{ lower triangular}$$

The system is then solved with two cheap triangular substitutions:

$$\mathbf{C} \mathbf{y} = -\mathbf{J} \longrightarrow \mathbf{C}^\top \Delta \mathbf{x} = \mathbf{y}$$

The **factorization** is the expensive part.

These are very expensive operations (no Real-Time)

**Idea of PD:** restrict the energies to a special form, so that the system matrix is **constant** and **SPD**, then **factorized once** and reused forever.

# Projective Dynamics (PD)

## Projective Dynamics: Fusing Constraint Projections for Fast Simulation

Sofien Bouaziz\*  
EPFL

Sebastian Martin†  
VM Research

Tiantian Liu‡  
University of Pennsylvania

Ladislav Kavan§  
University of Pennsylvania

Mark Pauly¶  
EPFL



[ <https://dl.acm.org/doi/10.1145/2601097.2601116> ]

## | Projective Dynamics (PD)

Each element energy is the squared distance between a linear function of the positions and a set of admissible configurations:

$$U_e(\mathbf{x}) = \min_{\mathbf{p}_e \in \mathcal{M}_e} \frac{k_e}{2} \left\| \mathbf{G}_e \mathbf{x} - \mathbf{p}_e \right\|^2$$

- $\mathbf{G}_e$  : **constant** linear operator (selects and combines the element's particles)
- $\mathcal{M}_e$  : **constraint manifold**, the set of undeformed (zero-energy) configurations
- $\mathbf{p}_e$  : auxiliary variable, the **projection** of  $\mathbf{G}_e \mathbf{x}$  onto  $\mathcal{M}_e$

# | Projective Dynamics (PD)

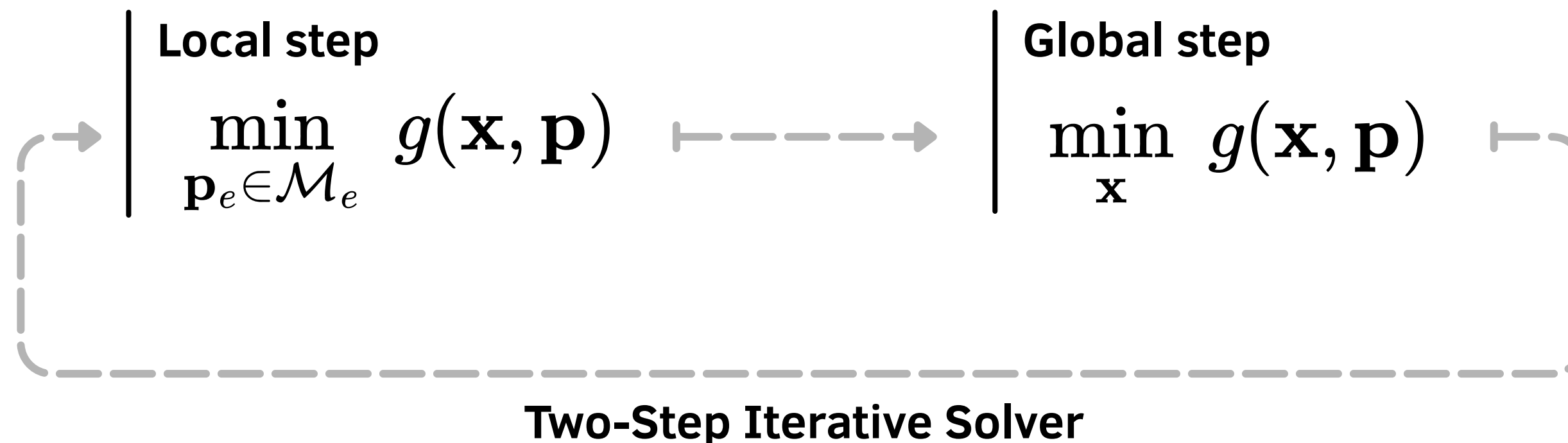
**Augmented Energy:**

$$g(\mathbf{x}, \mathbf{p}) = \frac{1}{2h^2} (\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M} (\mathbf{x} - \tilde{\mathbf{x}}) + \sum_e \frac{k_e}{2} \|\mathbf{G}_e \mathbf{x} - \mathbf{p}_e\|^2$$

**Minimization Problem:**

$$\min_{\mathbf{x}, \mathbf{p}_e \in \mathcal{M}_e} g(\mathbf{x}, \mathbf{p})$$

We now minimize over two sets of variables, positions and projections. Instead of solving for both together, we alternate: minimize over one set while keeping the other fixed. This is alternating minimization, it turns the problem into a two-step iterative solver in which each step never increases the energy



## | Projective Dynamics (PD)

$$\min_{\mathbf{x}, \mathbf{p}_e \in \mathcal{M}_e} g(\mathbf{x}, \mathbf{p}) = \frac{1}{2h^2} (\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M} (\mathbf{x} - \tilde{\mathbf{x}}) + \sum_e \frac{k_e}{2} \|\mathbf{G}_e \mathbf{x} - \mathbf{p}_e\|^2$$

**Local step:** fix  $\mathbf{x}$ , minimize over each  $\mathbf{p}_e$ . Each is a small **independent projection**, done in **parallel**:

$$\min_{\mathbf{p}_e \in \mathcal{M}_e} g(\mathbf{x}, \mathbf{p}) \longrightarrow \mathbf{p}_e = \text{Proj}_{\mathcal{M}_e} (\mathbf{G}_e \mathbf{x}^{(k)})$$

**Global step:** fix  $\mathbf{p}_e$ , minimize over  $\mathbf{x}$ . The objective is quadratic, so we set its gradient to zero, which gives the linear system:

$$\underbrace{\left[ \frac{1}{h^2} \mathbf{M} + \sum_e k_e \mathbf{G}_e^\top \mathbf{G}_e \right]}_{\mathbf{L}_{\text{PD}} : \text{is constant and SPD}} \mathbf{x}^{(k+1)} = \underbrace{\frac{1}{h^2} \mathbf{M} \tilde{\mathbf{x}} + \sum_e k_e \mathbf{G}_e^\top \mathbf{p}_e}_{\mathbf{b}}$$

# | Projective Dynamics (PD)

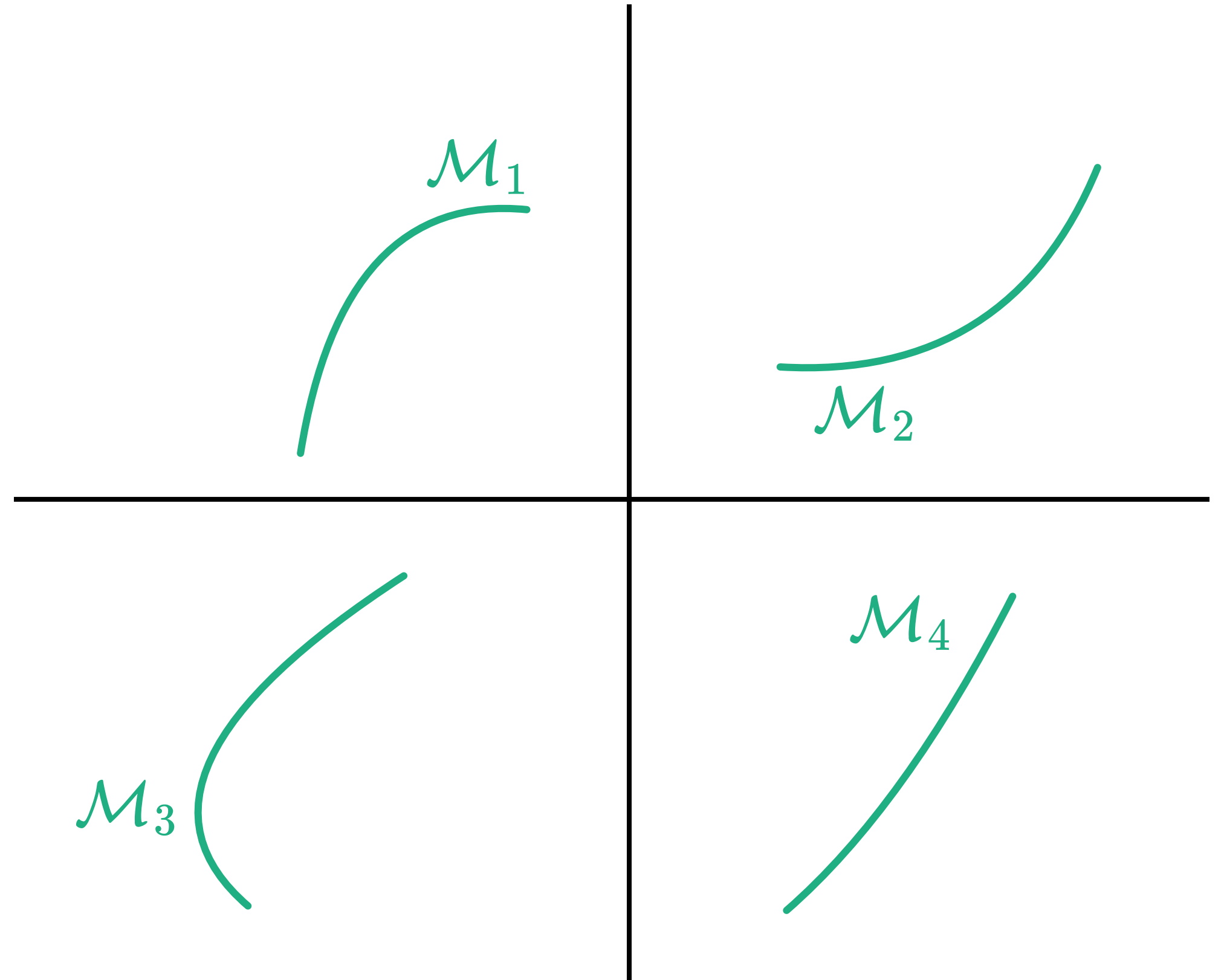
**for**  $k \in [0, 1, \dots, N - 1]$  **do**:

**for each element**  $e$  **do**:

    Compute  $\mathbf{G}_e \mathbf{x}^{(k)}$

$\mathbf{p}_e = \text{Proj}_{\mathcal{M}_e} (\mathbf{G}_e \mathbf{x}^{(k)})$

  Solve  $\mathbf{L}_{\text{PD}} \mathbf{x}^{(k+1)} = \mathbf{b}$



# | Projective Dynamics (PD)

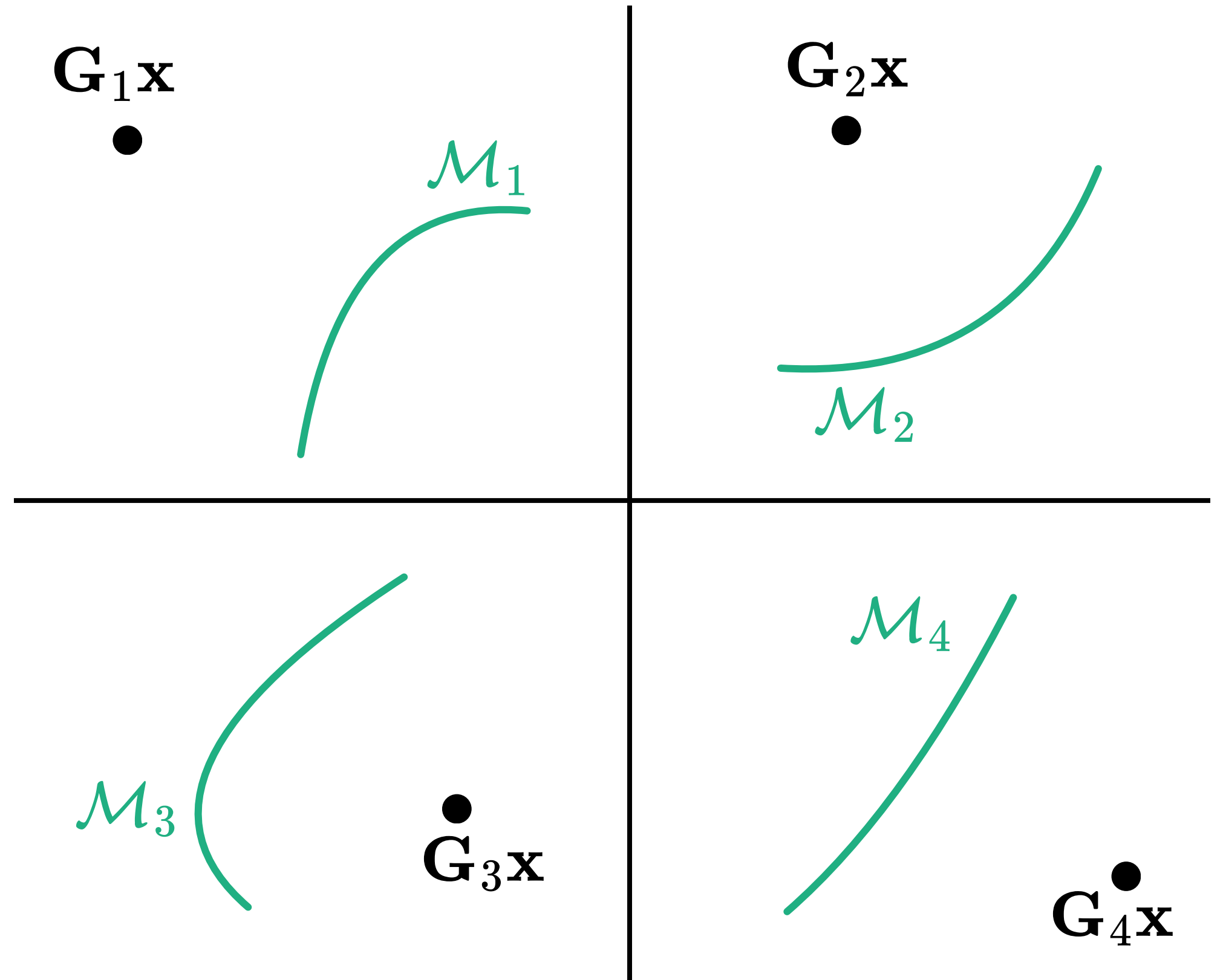
**for**  $k \in [0, 1, \dots, N - 1]$  **do**:

**for each element**  $e$  **do**:

Compute  $\mathbf{G}_e \mathbf{x}^{(k)}$

$\mathbf{p}_e = \text{Proj}_{\mathcal{M}_e} (\mathbf{G}_e \mathbf{x}^{(k)})$

Solve  $\mathbf{L}_{\text{PD}} \mathbf{x}^{(k+1)} = \mathbf{b}$



# | Projective Dynamics (PD)

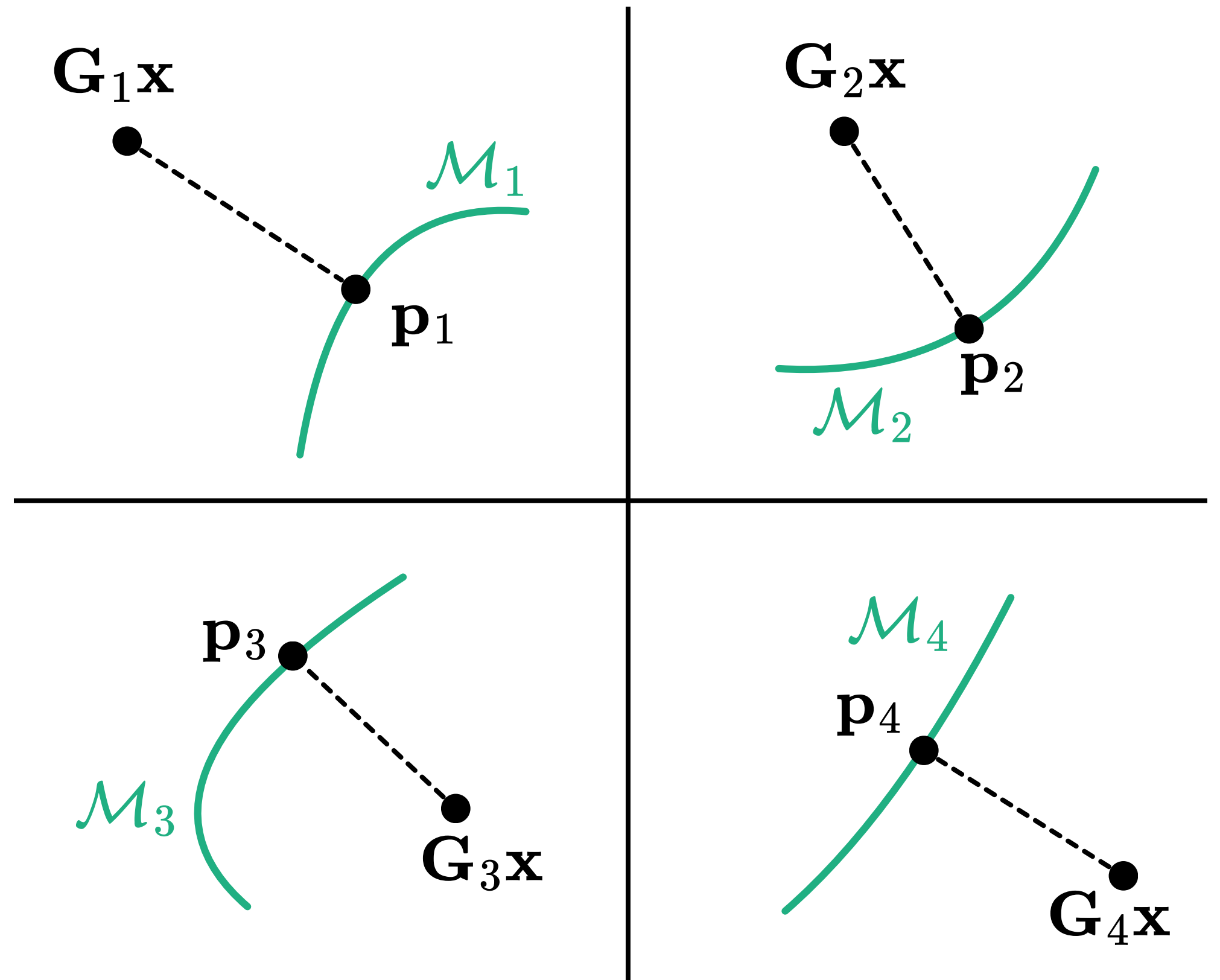
**for**  $k \in [0, 1, \dots, N - 1]$  **do**:

**for each element**  $e$  **do**:

Compute  $\mathbf{G}_e \mathbf{x}^{(k)}$

$$\mathbf{p}_e = \text{Proj}_{\mathcal{M}_e} (\mathbf{G}_e \mathbf{x}^{(k)})$$

Solve  $\mathbf{L}_{\text{PD}} \mathbf{x}^{(k+1)} = \mathbf{b}$



# | Projective Dynamics (PD)

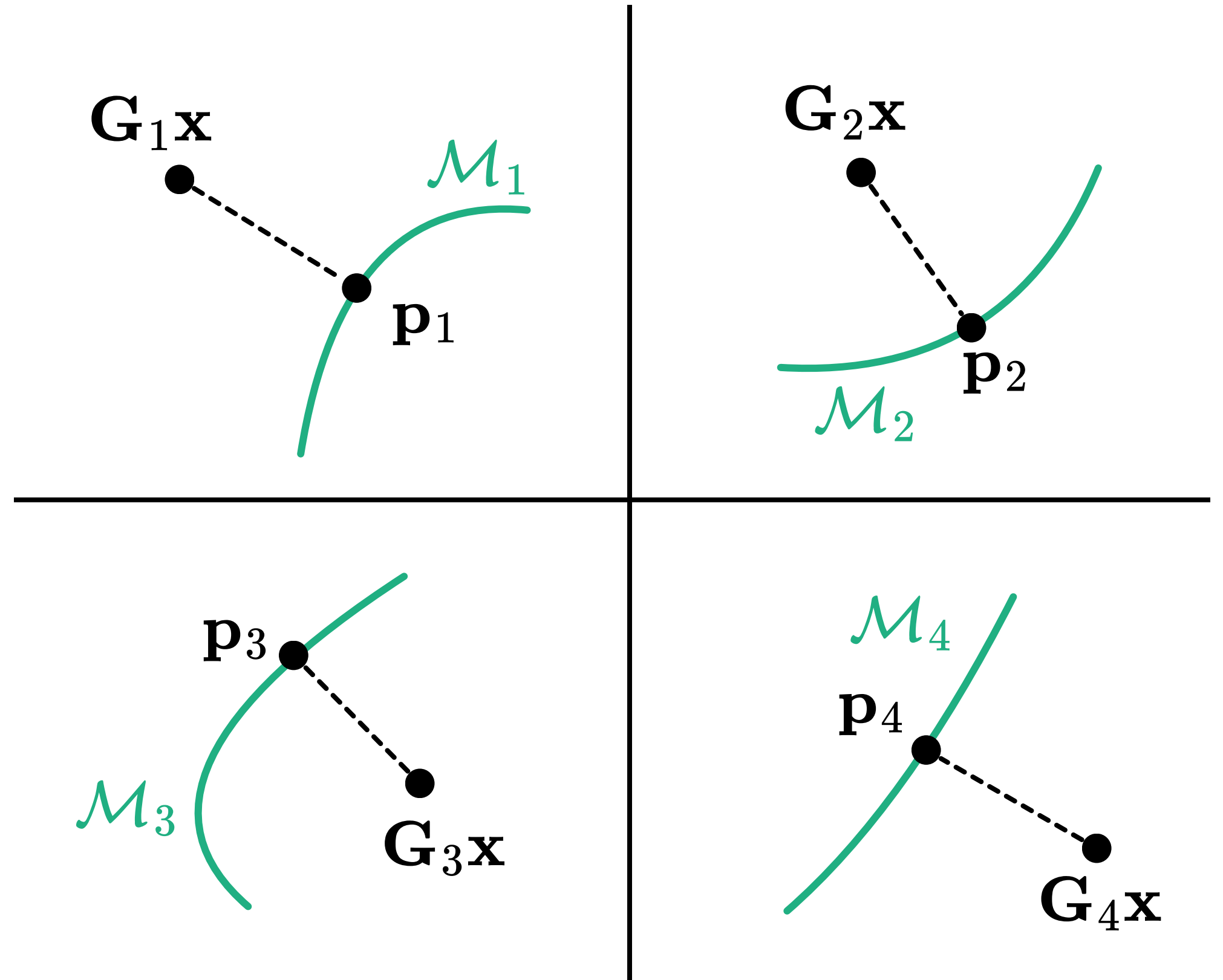
**for**  $k \in [0, 1, \dots, N - 1]$  **do**:

**for each element**  $e$  **do**:

Compute  $\mathbf{G}_e \mathbf{x}^{(k)}$

$$\mathbf{p}_e = \text{Proj}_{\mathcal{M}_e} (\mathbf{G}_e \mathbf{x}^{(k)})$$

Solve  $\mathbf{L}_{\text{PD}} \mathbf{x}^{(k+1)} = \mathbf{b}$



# | Projective Dynamics (PD)

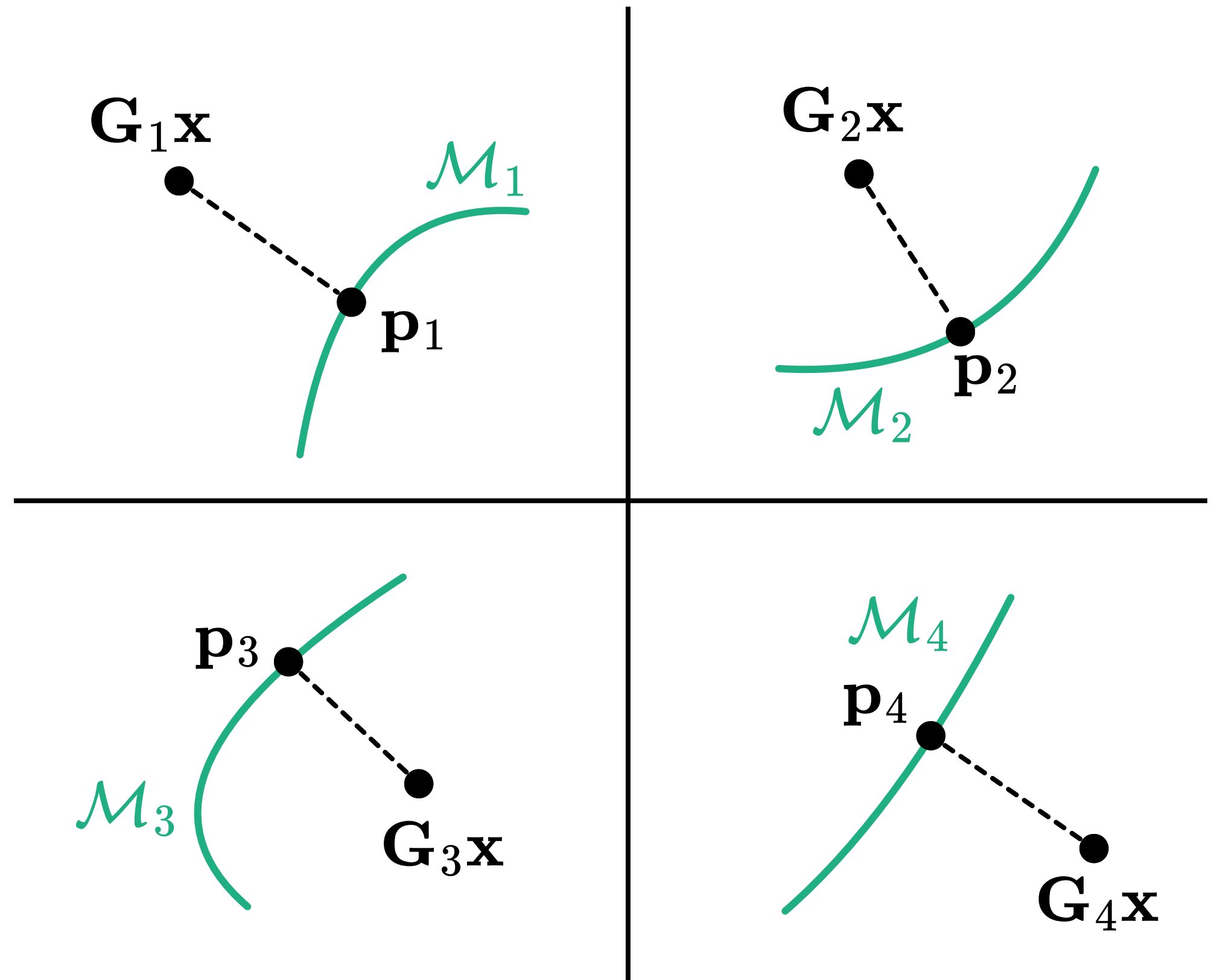
**for**  $k \in [0, 1, \dots, N - 1]$  **do:**

**for each element**  $e$  **do:**

Compute  $\mathbf{G}_e \mathbf{x}^{(k)}$

$$\mathbf{p}_e = \text{Proj}_{\mathcal{M}_e} (\mathbf{G}_e \mathbf{x}^{(k)})$$

Solve  $\mathbf{L}_{\text{PD}} \mathbf{x}^{(k+1)} = \mathbf{b}$



# | Projective Dynamics (PD)

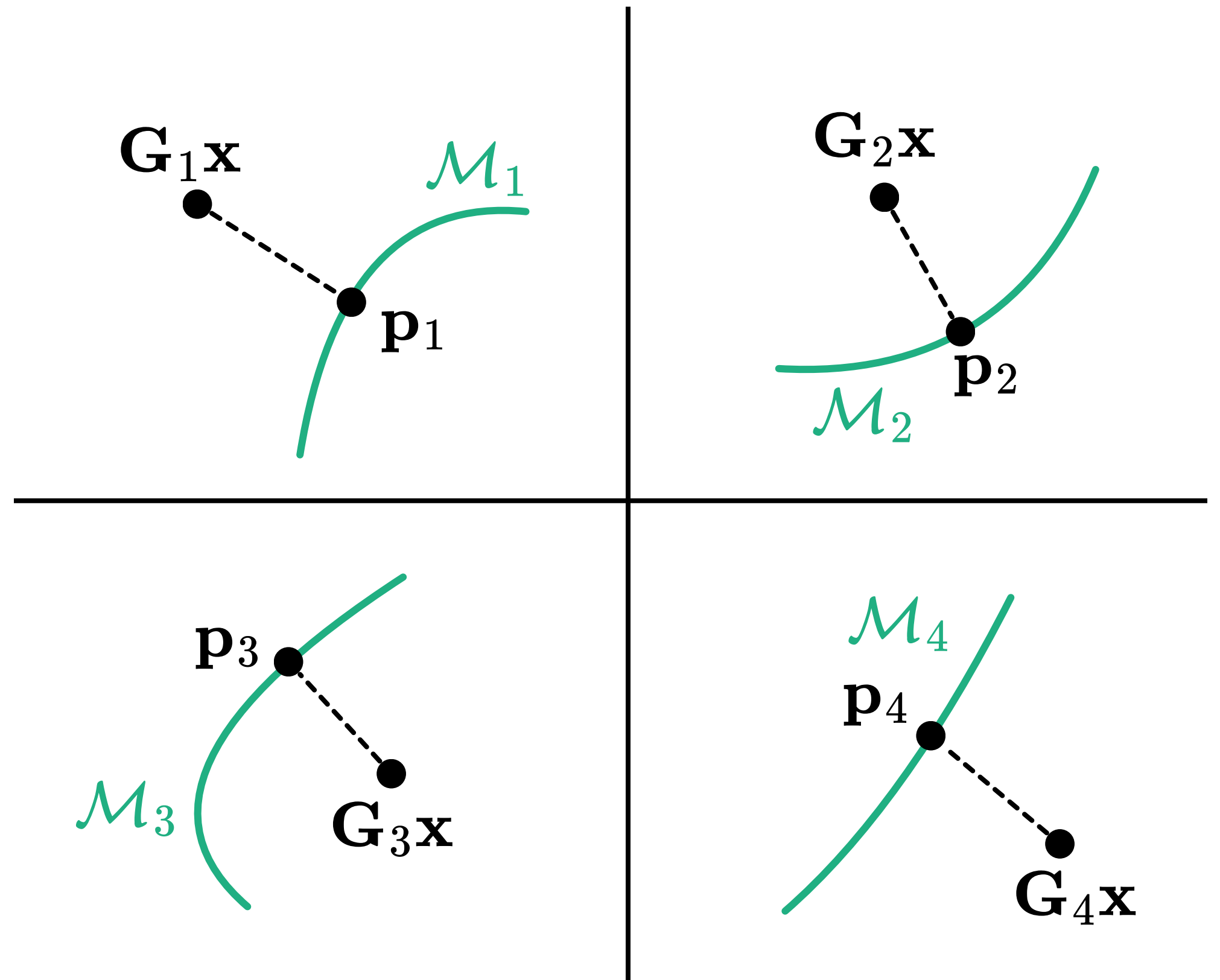
**for**  $k \in [0, 1, \dots, N - 1]$  **do:**

**for each element**  $e$  **do:**

Compute  $\mathbf{G}_e \mathbf{x}^{(k)}$

$$\mathbf{p}_e = \text{Proj}_{\mathcal{M}_e} (\mathbf{G}_e \mathbf{x}^{(k)})$$

$$\text{Solve } \mathbf{L}_{\text{PD}} \mathbf{x}^{(k+1)} = \mathbf{b}$$



# | Projective Dynamics (PD)

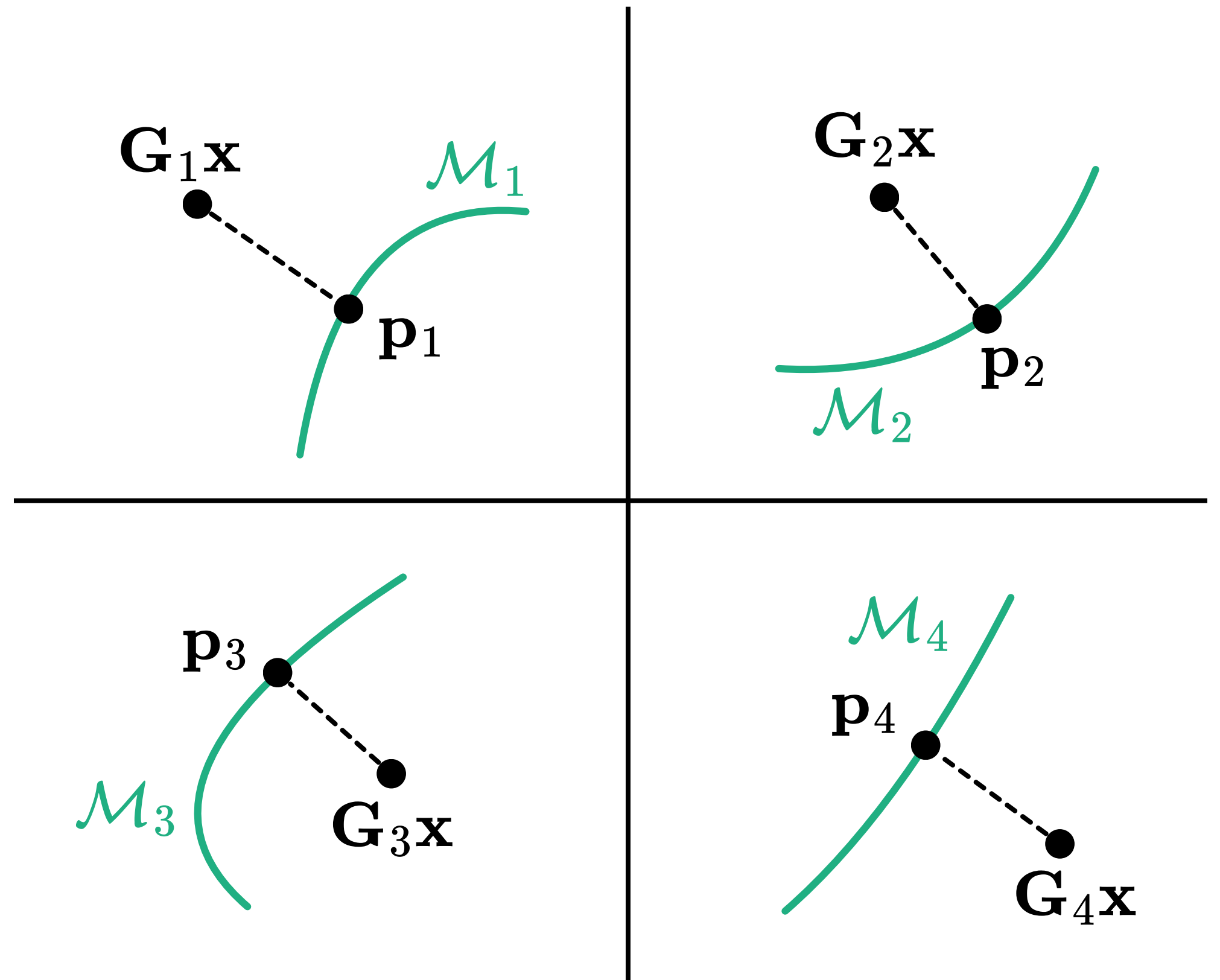
**for**  $k \in [0, 1, \dots, N - 1]$  **do**:

**for each element**  $e$  **do**:

Compute  $\mathbf{G}_e \mathbf{x}^{(k)}$

$$\mathbf{p}_e = \text{Proj}_{\mathcal{M}_e} (\mathbf{G}_e \mathbf{x}^{(k)})$$

Solve  $\mathbf{L}_{\text{PD}} \mathbf{x}^{(k+1)} = \mathbf{b}$



# | Projective Dynamics (PD)

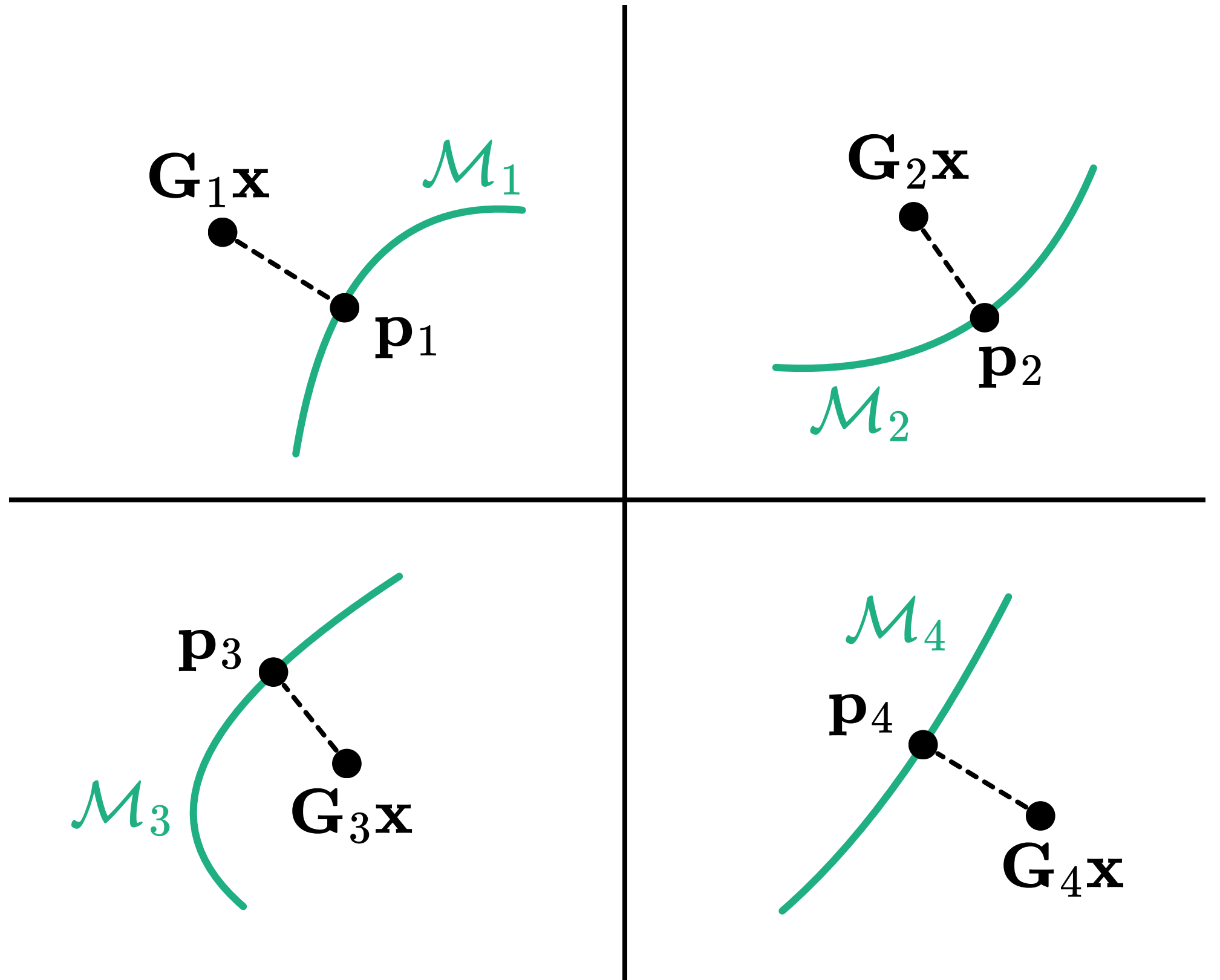
**for**  $k \in [0, 1, \dots, N - 1]$  **do**:

**for each element**  $e$  **do**:

        Compute  $\mathbf{G}_e \mathbf{x}^{(k)}$

$\mathbf{p}_e = \text{Proj}_{\mathcal{M}_e} (\mathbf{G}_e \mathbf{x}^{(k)})$

    Solve  $\mathbf{L}_{\text{PD}} \mathbf{x}^{(k+1)} = \mathbf{b}$



# | Projective Dynamics (PD)

**function** PROJECTIVEDYNAMICS( $\mathbf{x}^-, \mathbf{v}^-$ )

1:  $\tilde{\mathbf{x}} \leftarrow \mathbf{x}^- + h \mathbf{v}^-$

2: Initialize  $\mathbf{x}^{(0)}$

3: **for**  $k = 0, 1, \dots, N - 1$  **do**

4:   **for** each element  $e$  **in parallel do**

5:      $\mathbf{p}_e \leftarrow \text{PROJ}_{\mathcal{M}_e}(\mathbf{G}_e \mathbf{x}^{(k)})$

6:    $\mathbf{b} \leftarrow \frac{1}{h^2} \mathbf{M} \tilde{\mathbf{x}} + \sum_e k_e \mathbf{G}_e^\top \mathbf{p}_e$

7:   **SOLVE**  $\mathbf{L}_{\text{PD}} \mathbf{x}^{(k+1)} = \mathbf{b}$

8:  $\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$

9:  $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$

**end function**

## Local step

Each element is projected onto its constraint set independently and in parallel.

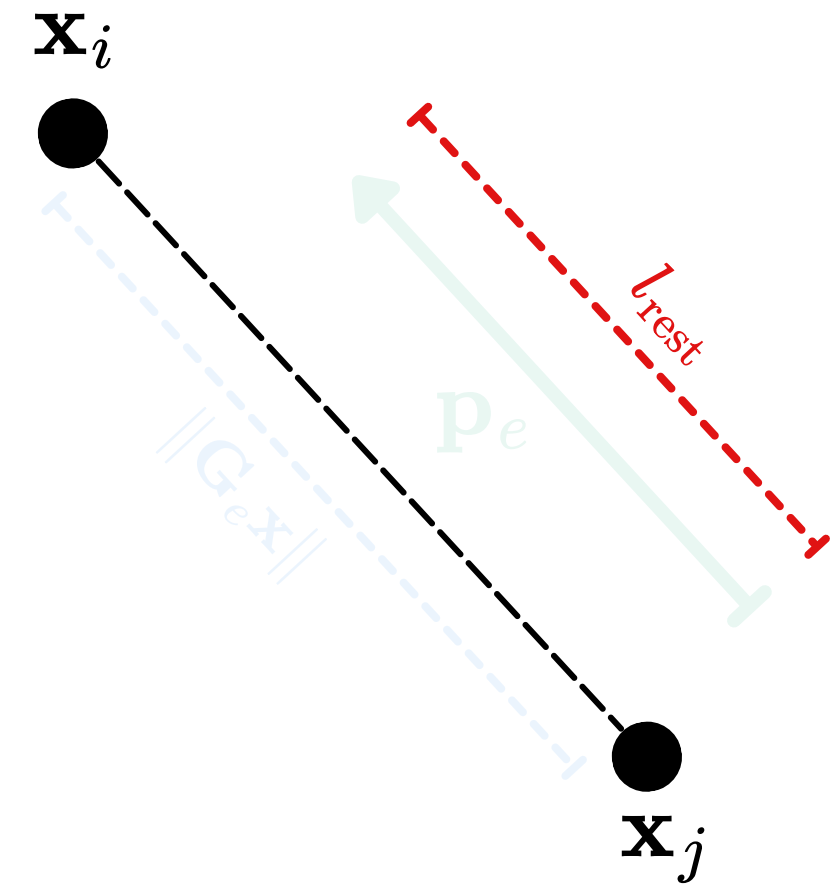
## Global step

Here, the linear solve is fast because the constant system matrix  $\mathbf{L}_{\text{PD}}$  has already been factorized once, at initialization.

# | Projective Dynamics (PD) Spring Constraint

Spring:

$i, j$  : indices of particles     $l_{\text{rest}}$  : rest length     $k$  : stiffness



# Projective Dynamics (PD) Spring Constraint

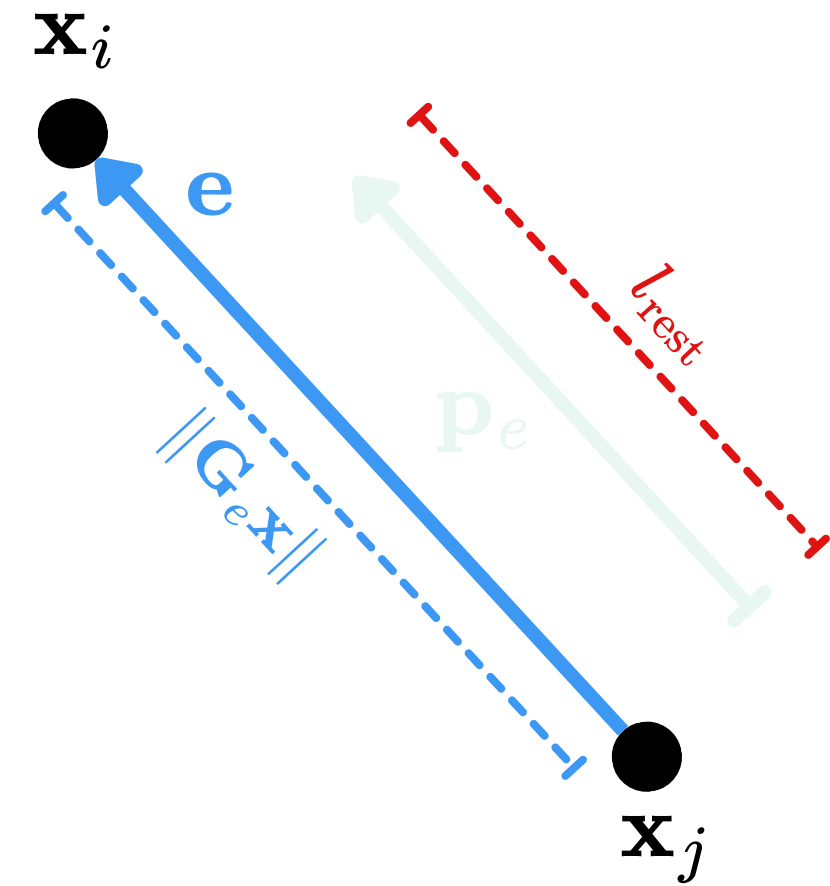
Spring:

$i, j$  : indices of particles     $l_{\text{rest}}$  : rest length     $k$  : stiffness

Linear Operator:

$$\mathbf{G}_e \in \mathbb{R}^{3 \times 3n}$$

$$\mathbf{G}_e \mathbf{x} = \begin{bmatrix} \cdots & \overset{i}{\mathbf{I}} & \cdots & \overset{j}{-\mathbf{I}} & \cdots \end{bmatrix}^\top \begin{bmatrix} \vdots \\ \mathbf{x}_i \\ \vdots \\ \mathbf{x}_j \\ \vdots \end{bmatrix} = \mathbf{x}_i - \mathbf{x}_j = \mathbf{e}$$



# Projective Dynamics (PD) Spring Constraint

Spring:

$i, j$  : indices of particles    $l_{\text{rest}}$  : rest length    $k$  : stiffness

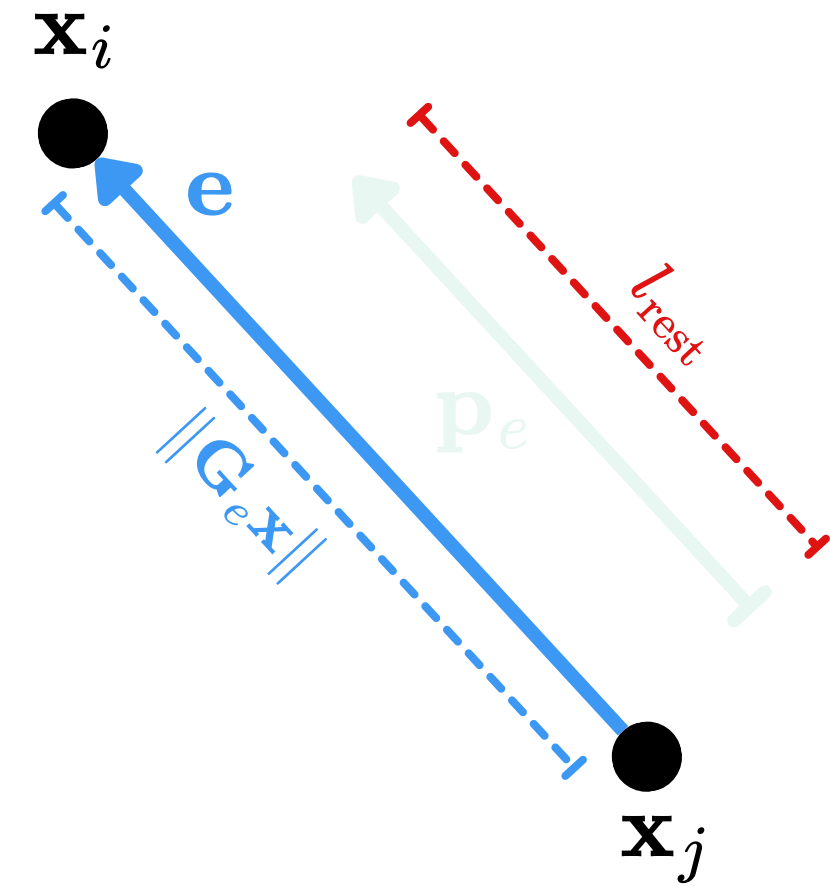
Linear Operator:

$$\mathbf{G}_e = \begin{bmatrix} \cdots & \overset{i}{\mathbf{I}} & \cdots & \overset{j}{-\mathbf{I}} & \cdots \end{bmatrix} \in \mathbb{R}^{3 \times 3n}$$

$$\mathbf{G}_e \mathbf{x} = \mathbf{x}_i - \mathbf{x}_j = \mathbf{e}$$

Constraint Manifold:

$$\mathcal{M}_e = \{ \mathbf{p} \in \mathbb{R}^3 : \|\mathbf{p}\| = l_{\text{rest}} \}$$



# Projective Dynamics (PD) Spring Constraint

**Spring:**

$i, j$  : indices of particles     $l_{\text{rest}}$  : rest length     $k$  : stiffness

**Linear Operator:**

$$\mathbf{G}_e = \begin{bmatrix} \cdots & \overset{i}{\mathbf{I}} & \cdots & \overset{j}{-\mathbf{I}} & \cdots \end{bmatrix} \in \mathbb{R}^{3 \times 3n}$$

$$\mathbf{G}_e \mathbf{x} = \mathbf{x}_i - \mathbf{x}_j = \mathbf{e}$$

**Constraint Manifold:**

$$\mathcal{M}_e = \left\{ \mathbf{p} \in \mathbb{R}^3 : \|\mathbf{p}\| = l_{\text{rest}} \right\}$$

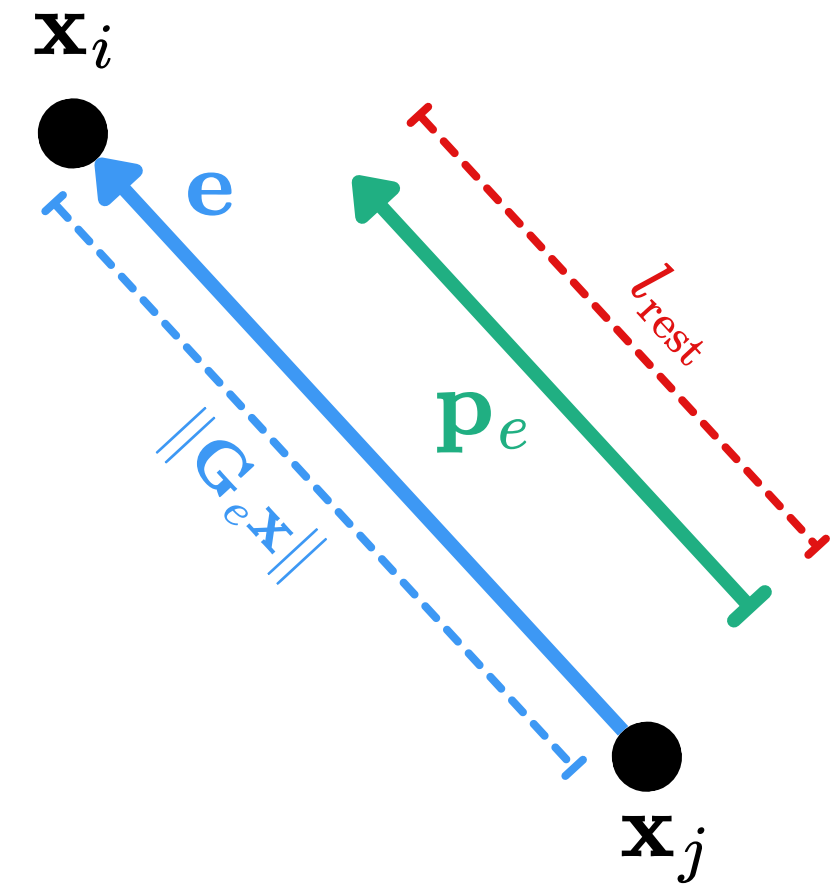
**Local Step (Projection):**

$$\mathbf{p}_e = l_{\text{rest}} \frac{\mathbf{G}_e \mathbf{x}}{\|\mathbf{G}_e \mathbf{x}\|}$$

**Global Step:**

$$\mathbf{L}_{\text{PD}} += k_e \mathbf{G}_e^\top \mathbf{G}_e$$

$$\mathbf{b} += k_e \mathbf{G}_e^\top \mathbf{p}_e$$



# Projective Dynamics (PD) Spring Constraint

**Spring:**

$i, j$  : indices of particles     $l_{\text{rest}}$  : rest length     $k$  : stiffness

**Linear Operator:**

$$\mathbf{G}_e = \begin{bmatrix} & i & & j & \\ \cdots & \mathbf{I} & \cdots & -\mathbf{I} & \cdots \end{bmatrix} \in \mathbb{R}^{3 \times 3n}$$

$$\mathbf{G}_e \mathbf{x} = \mathbf{x}_i - \mathbf{x}_j = \mathbf{e}$$

**Constraint Manifold:**

$$\mathcal{M}_e = \{ \mathbf{p} \in \mathbb{R}^3 : \|\mathbf{p}\| = l_{\text{rest}} \}$$

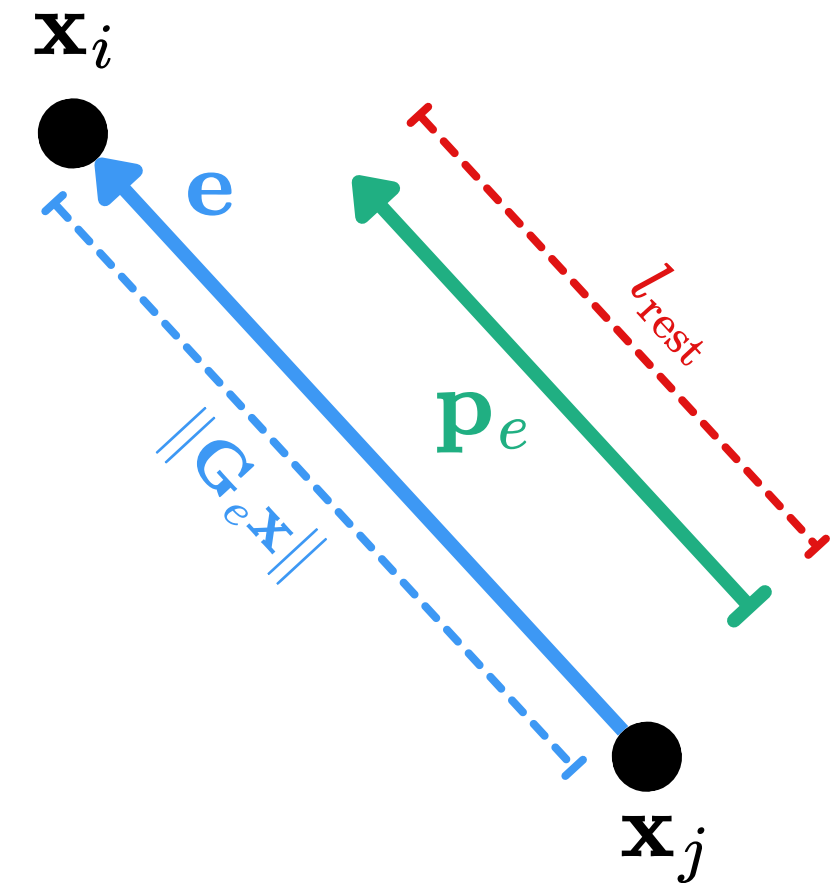
**Local Step (Projection):**

$$\mathbf{p}_e = l_{\text{rest}} \frac{\mathbf{G}_e \mathbf{x}}{\|\mathbf{G}_e \mathbf{x}\|}$$

**Global Step:**

$$\mathbf{L}_{\text{PD}} += k_e \mathbf{G}_e^\top \mathbf{G}_e$$

$$\mathbf{b} += k_e \mathbf{G}_e^\top \mathbf{p}_e$$



$$\begin{matrix} & i & & j & \\ & \vdots & & \vdots & \\ i & \cdots & \mathbf{I} & \cdots & -\mathbf{I} & \cdots \\ & \vdots & & \vdots & & \\ j & \cdots & -\mathbf{I} & \cdots & \mathbf{I} & \cdots \\ & \vdots & & \vdots & & \end{matrix} \in \mathbb{R}^{3n \times 3n}$$

$$\begin{bmatrix} \cdots & \mathbf{p}_e & \cdots & -\mathbf{p}_e & \cdots \end{bmatrix}^\top \in \mathbb{R}^{3n}$$

A thin vertical black line is positioned on the left side of the slide, extending from the top of the text area to the bottom.

# **Differentiating Through Variational Implicit Euler Solver**

# | Differentiating Through Variational Implicit Euler Solver

The loss is measured on the simulated trajectory. We know the gradient of the loss after a step and want it before the step:

$$\text{known: } \frac{\partial \phi}{\partial \mathbf{x}^+} \longrightarrow \text{wanted: } \frac{\partial \phi}{\partial \mathbf{x}^-}$$

Repeating this step by step, from the last to first frame, gives the gradient with respect to the initial state:

$$\frac{\partial \phi}{\partial \mathbf{x}_T} \xrightarrow{\text{step } T} \frac{\partial \phi}{\partial \mathbf{x}_{T-1}} \xrightarrow{\text{step } T-1} \frac{\partial \phi}{\partial \mathbf{x}_{T-2}} \longrightarrow \dots \longrightarrow \frac{\partial \phi}{\partial \mathbf{x}_0}$$

The building block we would like to compute then is:

$$\frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-} \xrightarrow{\text{which through chain rule gives us:}} \frac{\partial \phi}{\partial \mathbf{x}^-} = \frac{\partial \phi}{\partial \mathbf{x}^+} \frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-}$$

# | Differentiating Through Variational Implicit Euler Solver

Energy and its Derivatives:

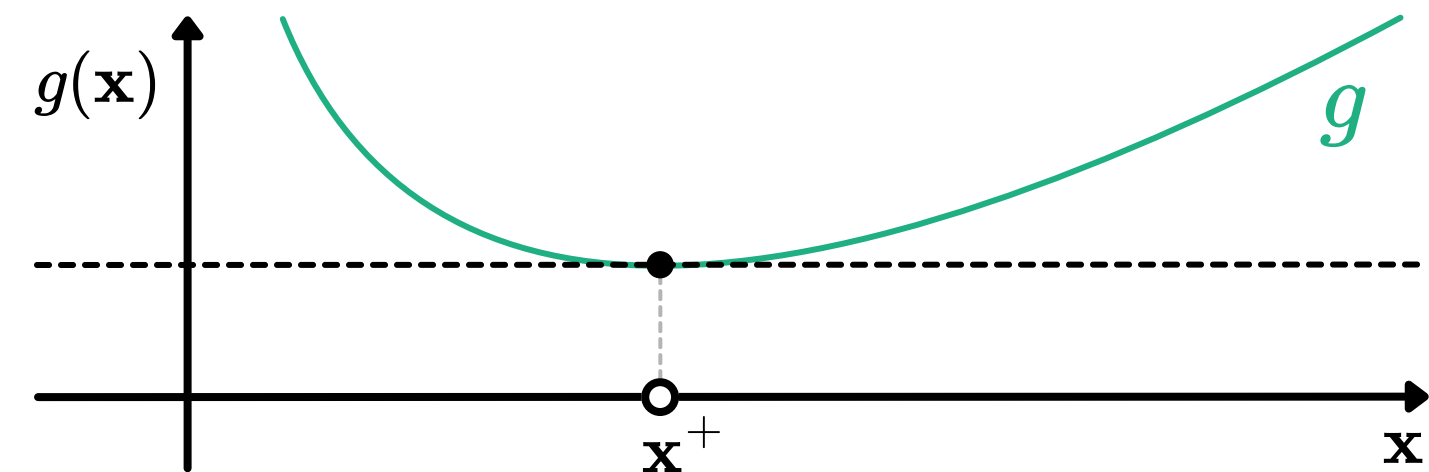
$$g(\mathbf{x}) = \frac{1}{2h^2} (\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M} (\mathbf{x} - \tilde{\mathbf{x}}) + U(\mathbf{x})$$

$$\mathbf{J} = \nabla g(\mathbf{x}) = \frac{1}{h^2} \mathbf{M} (\mathbf{x} - \tilde{\mathbf{x}}) + \nabla U(\mathbf{x})$$

$$\mathbf{H} = \nabla^2 g(\mathbf{x}) = \frac{1}{h^2} \mathbf{M} + \nabla^2 U(\mathbf{x})$$

At the end of the step the converged solution  $\mathbf{x}^+$  satisfy the **optimality condition**:

$$\nabla g(\mathbf{x}^+) = \mathbf{0}$$



This holds at convergence whatever solver was used (Newton, PD, ... )

We differentiate this condition (**implicit differentiation**), not the solver's iterations.

# | Differentiating Through Variational Implicit Euler Solver

Energy and its Derivatives:

$$g(\mathbf{x}) = \frac{1}{2h^2} (\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M} (\mathbf{x} - \tilde{\mathbf{x}}) + U(\mathbf{x})$$

$$\mathbf{J} = \nabla g(\mathbf{x}) = \frac{1}{h^2} \mathbf{M} (\mathbf{x} - \tilde{\mathbf{x}}) + \nabla U(\mathbf{x})$$

$$\mathbf{H} = \nabla^2 g(\mathbf{x}) = \frac{1}{h^2} \mathbf{M} + \nabla^2 U(\mathbf{x})$$

Differentiating the optimality condition with respect to  $\mathbf{x}^-$  gives:

$$\underbrace{\left[ \frac{1}{h^2} \mathbf{M} + \nabla^2 U(\mathbf{x}^+) \right]}_{\mathbf{H} = \nabla^2 g(\mathbf{x}^+)} \frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-} - \frac{1}{h^2} \mathbf{M} = \mathbf{0} \quad \Rightarrow \quad \frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-} = \frac{1}{h^2} \mathbf{H}^{-1} \mathbf{M}$$

Then the chain rule gives:

$$\frac{\partial \phi}{\partial \mathbf{x}^-} = \frac{\partial \phi}{\partial \mathbf{x}^+} \frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-} = \frac{1}{h^2} \frac{\partial \phi}{\partial \mathbf{x}^+} \mathbf{H}^{-1} \mathbf{M}$$

computed once at convergence

# | Differentiating Through Variational Implicit Euler Solver

$$\frac{\partial \phi}{\partial \mathbf{x}^-} = \frac{1}{h^2} \underbrace{\frac{\partial \phi}{\partial \mathbf{x}^+} \mathbf{H}^{-1}}_{\mathbf{z}^\top} \mathbf{M}$$

We never compute the dense matrix  $\mathbf{H}^{-1}$ . Instead, we group the first two factors into the **adjoint vector**  $\mathbf{z}$ , which comes from a single linear solve:

$$\mathbf{z}^\top = \frac{\partial \phi}{\partial \mathbf{x}^+} \mathbf{H}^{-1} \iff \mathbf{H} \mathbf{z} = \left( \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top \longrightarrow \text{Single Linear System Solve}$$

Then both positions and velocities gradients are cheap matrix–vector products:

$$\frac{\partial \phi}{\partial \mathbf{x}^-} = \frac{1}{h^2} \mathbf{z}^\top \mathbf{M} \qquad \frac{\partial \phi}{\partial \mathbf{v}^-} = \frac{1}{h} \mathbf{z}^\top \mathbf{M}$$

# Differentiating Through Variational Implicit Euler Solver

**function** NEWTONSTEP( $\mathbf{x}^-$ ,  $\mathbf{v}^-$ )

```
1: Initialize  $\mathbf{x}$ 
2: for  $k = 0, 1, \dots, N - 1$  do
3:   BUILD  $\mathbf{J}$ ,  $\mathbf{H}$ 
4:   SOLVE  $\mathbf{H} \Delta \mathbf{x} = -\mathbf{J}$ 
5:    $\mathbf{x} \leftarrow \mathbf{x} + \Delta \mathbf{x}$ 
6:  $\mathbf{x}^+ \leftarrow \mathbf{x}$ 
7:  $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$ 
```

**end function**

**Forward:**

N iterations, and each one builds and solves with a new Hessian.

**function** BACKWARDSTEP( $\mathbf{x}^+$ ,  $\frac{\partial \phi}{\partial \mathbf{x}^+}$ )

```
1: BUILD  $\mathbf{H} = \nabla^2 g(\mathbf{x}^+)$ 
2: SOLVE  $\mathbf{H} \mathbf{z} = \left( \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top$ 
3:  $\frac{\partial \phi}{\partial \mathbf{x}^-} \leftarrow \frac{1}{h^2} \mathbf{z}^\top \mathbf{M}$ 
4:  $\frac{\partial \phi}{\partial \mathbf{v}^-} \leftarrow \frac{1}{h} \mathbf{z}^\top \mathbf{M}$ 
```

**end function**

**Backward:**

One solve with the same Hessian built at convergence, costs about one Newton iteration.

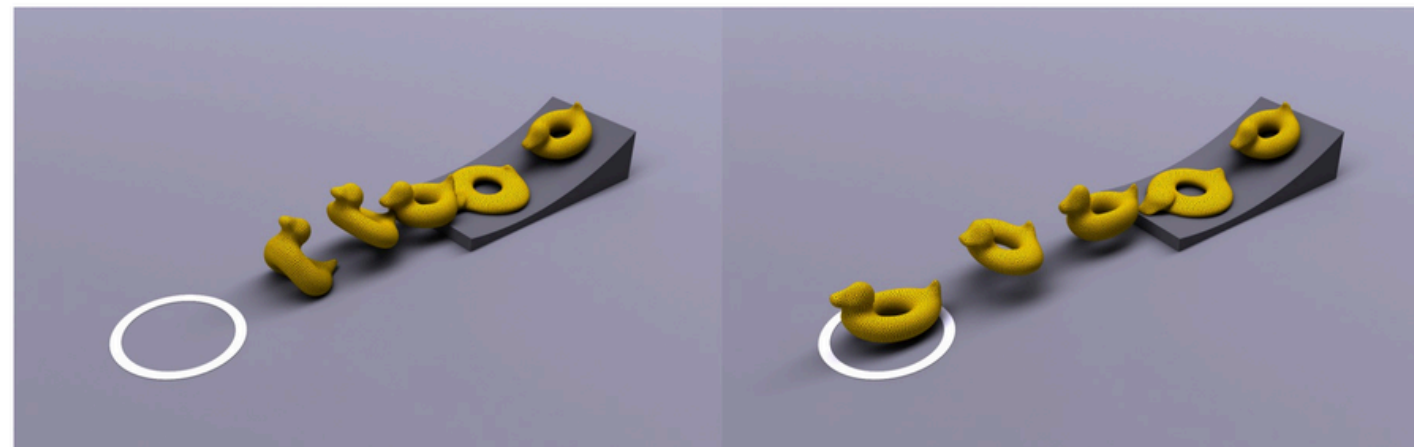
**The Question then is:**

Since in PD we never construct the true Hessian  $\mathbf{H}$  explicitly, how do we compute the adjoint vector?

# Differentiable Projective Dynamics: DiffPD

DiffPD: Differentiable Projective Dynamics

TAO DU, KUI WU, PINGCHUAN MA, SEBASTIEN WAH, and ANDREW SPIELBERG, MIT CSAIL  
DANIELA RUS and WOJCIECH MATUSIK, MIT CSAIL



[ <https://diffpd.csail.mit.edu> ]

# | Differentiable Projective Dynamics: DiffPD

We should find a way to compute:  $\mathbf{H}_{\text{PD}} = \nabla^2 g_{\text{PD}}(\mathbf{x})$  We need it for:  
 $\mathbf{H} \mathbf{z} = \left( \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top$

PD energy is:  $g(\mathbf{x}, \mathbf{p}) = \frac{1}{2h^2} (\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M} (\mathbf{x} - \tilde{\mathbf{x}}) + \underbrace{\sum \frac{k_e}{2} \|\mathbf{G}_e \mathbf{x} - \mathbf{p}_e\|^2}_U$

But the projections really depends on the positions:

$$\mathbf{p}_e(\mathbf{x}) = \text{Proj}_{\mathcal{M}_e}(\mathbf{G}_e \mathbf{x}) \quad \mapsto \quad U(\mathbf{x}) = \sum \frac{k_e}{2} \|\mathbf{G}_e \mathbf{x} - \mathbf{p}_e(\mathbf{x})\|^2$$

From which we can derive the first and second derivatives:

$$\nabla U(\mathbf{x}) = \sum k_e \mathbf{G}_e^\top (\mathbf{G}_e \mathbf{x} - \mathbf{p}_e) \quad \nabla^2 U(\mathbf{x}) = \sum k_e \mathbf{G}_e^\top \mathbf{G}_e - \sum k_e \mathbf{G}_e^\top \frac{\partial \mathbf{p}_e}{\partial \mathbf{x}}$$

And then we can compute the full real PD Hessian:

$$\mathbf{H}_{\text{PD}} = \frac{1}{h^2} \mathbf{M} + \sum k_e \mathbf{G}_e^\top \mathbf{G}_e - \sum k_e \mathbf{G}_e^\top \frac{\partial \mathbf{p}_e}{\partial \mathbf{x}}$$

# | Differentiable Projective Dynamics: DiffPD

$$\boxed{\mathbf{H}_{\text{PD}}} = \underbrace{\frac{1}{h^2} \mathbf{M} + \sum k_e \mathbf{G}_e^\top \mathbf{G}_e}_{\mathbf{L}_{\text{PD}}} - \underbrace{\sum k_e \mathbf{G}_e^\top \frac{\partial \mathbf{p}_e}{\partial \mathbf{x}}}_{\Delta \mathbf{L}} = \boxed{\mathbf{L}_{\text{PD}} - \Delta \mathbf{L}}$$

- $\mathbf{L}_{\text{PD}}$  : constant, and already factorized in the forward simulation.
- $\Delta \mathbf{L}$  : contains all the nonlinearity; it is a sum of small per-element terms, computed in parallel.

$$\frac{\partial \mathbf{p}_e}{\partial \mathbf{x}} = \frac{\partial \text{Proj}_{\mathcal{M}_e}}{\partial (\underbrace{\mathbf{G}_e \mathbf{x}}_{\Gamma_e})} \mathbf{G}_e = \mathbf{\Gamma}_e \mathbf{G}_e \quad \Rightarrow \quad \Delta \mathbf{L} = \sum_e k_e \mathbf{G}_e^\top \mathbf{\Gamma}_e \mathbf{G}_e$$

## | Differentiable Projective Dynamics: DiffPD

Substituting the split into the adjoint system  $\mathbf{H} \mathbf{z} = \left( \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top$  we obtain:

$$(\mathbf{L}_{\text{PD}} - \Delta \mathbf{L}) \mathbf{z} = \left( \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top$$

Moving  $\Delta \mathbf{L} \mathbf{z}$  to the right-hand side and evaluating it at the previous iterate gives a **fixed-point iteration**:

$$\mathbf{L}_{\text{PD}} \mathbf{z}^{(k+1)} = \Delta \mathbf{L} \mathbf{z}^{(k)} + \left( \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top$$

This has the same local/global structure as forward PD:

**Local step:**

$$\Delta \mathbf{L} \mathbf{z}^{(k)} = \sum k_e \mathbf{G}_e^\top \frac{\partial \mathbf{p}_e}{\partial \mathbf{x}} \mathbf{z}^{(k)} \quad \text{Computed element by element and in parallel, using only the small projection Jacobians.}$$

**Global step:**

$$\text{Solve } \mathbf{L}_{\text{PD}} \mathbf{z}^{(k+1)} = \dots \quad \text{One back-substitution with the already factorized } \mathbf{L}_{\text{PD}} \text{ in the forward pass; no new matrix is assembled or factorized.}$$

# Differentiable Projective Dynamics: DiffPD

## Backward Algorithm

**function** DIFFPDBACKWARDSTEP( $\mathbf{x}^+$ ,  $\frac{\partial \phi}{\partial \mathbf{x}^+}$ )

1: **for** each element  $e$  **in parallel do**

2:      $\Gamma_e \leftarrow \frac{\partial \text{PROJ}_{\mathcal{M}_e}}{\partial (\mathbf{G}_e \mathbf{x})}$

Compute Projection derivatives **at convergence** that we constantly reuse during the Local-Global steps.

3: Initialize  $\mathbf{z}$

4: **for**  $k = 0, 1, \dots, K - 1$  **do**

5:      $\mathbf{b} \leftarrow \sum_e k_e \mathbf{G}_e^\top \Gamma_e \mathbf{G}_e \mathbf{z}$

**Local step**

6:     **SOLVE**  $\mathbf{L}_{\text{PD}} \mathbf{z} = \mathbf{b} + \left( \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top$

**Global step**

7:      $\frac{\partial \phi}{\partial \mathbf{x}^-} \leftarrow \frac{1}{h^2} \mathbf{z}^\top \mathbf{M}$

8:      $\frac{\partial \phi}{\partial \mathbf{v}^-} \leftarrow \frac{1}{h} \mathbf{z}^\top \mathbf{M}$

**end function**

# Differentiable Projective Dynamics: DiffPD

## Spring Constraint

### Spring:

$i, j$  : indices of particles     $l_{\text{rest}}$  : rest length     $k$  : stiffness

$$\mathbf{G}_e \mathbf{x} = \mathbf{x}_i - \mathbf{x}_j = \mathbf{e}$$

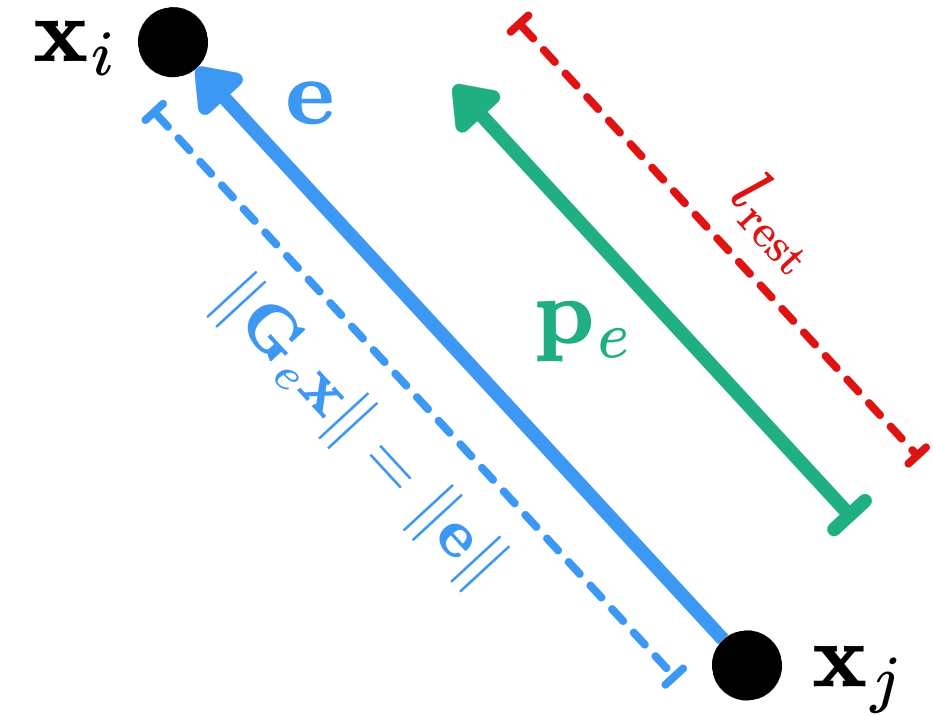
### Projection:

$$\mathbf{p}_e = \text{Proj}_{\mathcal{M}_e}(\mathbf{e}) = l_{\text{rest}} \hat{\mathbf{e}}, \quad \hat{\mathbf{e}} = \frac{\mathbf{e}}{\|\mathbf{e}\|}$$

### Projection Derivative:

$$\frac{\partial \hat{\mathbf{e}}}{\partial \mathbf{e}} = \frac{1}{\|\mathbf{e}\|} (\mathbf{I} - \hat{\mathbf{e}} \hat{\mathbf{e}}^\top)$$

$$\mathbf{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial \mathbf{e}} = \frac{l_{\text{rest}}}{\|\mathbf{e}\|} (\mathbf{I} - \hat{\mathbf{e}} \hat{\mathbf{e}}^\top) \in \mathbb{R}^{3 \times 3}$$



### Hessian Correction:

$$\Delta \mathbf{L}_e = k_e \mathbf{G}_e^\top \mathbf{\Gamma}_e \mathbf{G}_e = \begin{bmatrix} & i & & j & \\ & \vdots & & \vdots & \\ \cdots & \mathbf{\Gamma}_e & \cdots & -\mathbf{\Gamma}_e & \cdots \\ & \vdots & & \vdots & \\ \cdots & -\mathbf{\Gamma}_e & \cdots & \mathbf{\Gamma}_e & \cdots \\ & \vdots & & \vdots & \end{bmatrix} \begin{matrix} i \\ j \end{matrix}$$

$$\in \mathbb{R}^{3n \times 3n}$$

# Differentiable Projective Dynamics: DiffPD

## Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.  
At convergence it holds that:

$$\mathbf{L}_{\text{PD}}(\theta) \mathbf{x}^+ = \mathbf{b}(\mathbf{x}^+, \theta)$$

# Differentiable Projective Dynamics: DiffPD

## Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.  
At convergence it holds that:

$$\mathbf{L}_{\text{PD}}(\theta) \mathbf{x}^+ = \mathbf{b}(\mathbf{x}^+, \theta)$$

Then we:

- **Differentiate** the converged equation with respect to  $\theta$ .

$$\frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta} \mathbf{x}^+ + \mathbf{L}_{\text{PD}} \frac{d \mathbf{x}^+}{d \theta} = \frac{\partial \mathbf{b}}{\partial \mathbf{x}^+} \frac{d \mathbf{x}^+}{d \theta} + \frac{\partial \mathbf{b}}{\partial \theta}$$

# Differentiable Projective Dynamics: DiffPD

## Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.  
At convergence it holds that:

$$\mathbf{L}_{\text{PD}}(\theta) \mathbf{x}^+ = \mathbf{b}(\mathbf{x}^+, \theta)$$

Then we:

- **Differentiate** the converged equation with respect to  $\theta$ .
- **Collect** the unknown sensitivity.

$$\underbrace{\left( \mathbf{L}_{\text{PD}} - \frac{\partial \mathbf{b}}{\partial \mathbf{x}^+} \right)}_{\mathbf{L}_{\text{PD}} - \Delta \mathbf{L} = \mathbf{H}_{\text{PD}}} \frac{d \mathbf{x}^+}{d \theta} = \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta} \mathbf{x}^+$$

# Differentiable Projective Dynamics: DiffPD

## Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.  
At convergence it holds that:

$$\mathbf{L}_{\text{PD}}(\theta) \mathbf{x}^+ = \mathbf{b}(\mathbf{x}^+, \theta)$$

Then we:

- **Differentiate** the converged equation with respect to  $\theta$ .
- **Collect** the unknown sensitivity.

$$\underbrace{\left( \mathbf{L}_{\text{PD}} - \frac{\partial \mathbf{b}}{\partial \mathbf{x}^+} \right)}_{\mathbf{L}_{\text{PD}} - \Delta \mathbf{L} = \mathbf{H}_{\text{PD}}} \frac{d \mathbf{x}^+}{d \theta} = \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta} \mathbf{x}^+$$

- **Recognize** the same adjoint vector we already computed in the chain rule:

$$\frac{d \phi}{d \theta} = \frac{\partial \phi}{\partial \mathbf{x}^+} \frac{d \mathbf{x}^+}{d \theta}$$

# Differentiable Projective Dynamics: DiffPD

## Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.  
At convergence it holds that:

$$\mathbf{L}_{\text{PD}}(\theta) \mathbf{x}^+ = \mathbf{b}(\mathbf{x}^+, \theta)$$

Then we:

- **Differentiate** the converged equation with respect to  $\theta$ .
- **Collect** the unknown sensitivity.

$$\underbrace{\left( \mathbf{L}_{\text{PD}} - \frac{\partial \mathbf{b}}{\partial \mathbf{x}^+} \right)}_{\mathbf{L}_{\text{PD}} - \Delta \mathbf{L} = \mathbf{H}_{\text{PD}}} \frac{d \mathbf{x}^+}{d \theta} = \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta} \mathbf{x}^+$$

- **Recognize** the same adjoint vector we already computed in the chain rule:

$$\frac{d \phi}{d \theta} = \frac{\partial \phi}{\partial \mathbf{x}^+} \frac{d \mathbf{x}^+}{d \theta} = \underbrace{\frac{\partial \phi}{\partial \mathbf{x}^+} \mathbf{H}_{\text{PD}}^{-1}}_{\mathbf{z}^\top} \left( \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta} \mathbf{x}^+ \right)$$

# Differentiable Projective Dynamics: DiffPD

## Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.  
At convergence it holds that:

$$\mathbf{L}_{\text{PD}}(\theta) \mathbf{x}^+ = \mathbf{b}(\mathbf{x}^+, \theta)$$

Then we:

- **Differentiate** the converged equation with respect to  $\theta$ .
- **Collect** the unknown sensitivity.

$$\underbrace{\left( \mathbf{L}_{\text{PD}} - \frac{\partial \mathbf{b}}{\partial \mathbf{x}^+} \right)}_{\mathbf{L}_{\text{PD}} - \Delta \mathbf{L} = \mathbf{H}_{\text{PD}}} \frac{d \mathbf{x}^+}{d \theta} = \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta} \mathbf{x}^+$$

- **Recognize** the same adjoint vector we already computed in the chain rule:

$$\boxed{\frac{d \phi}{d \theta}} = \frac{\partial \phi}{\partial \mathbf{x}^+} \frac{d \mathbf{x}^+}{d \theta} = \underbrace{\frac{\partial \phi}{\partial \mathbf{x}^+} \mathbf{H}_{\text{PD}}^{-1}}_{\mathbf{z}^\top} \left( \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta} \mathbf{x}^+ \right) = \boxed{\mathbf{z}^\top \left( \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta} \mathbf{x}^+ \right)}$$

# Differentiable Projective Dynamics: DiffPD

Gradient with Respect to a General Parameter

$$\mathbf{L}_{\text{PD}} = \frac{1}{h^2} \mathbf{M} + \sum_e k_e \mathbf{G}_e^\top \mathbf{G}_e$$

$$\mathbf{b} = \frac{1}{h^2} \mathbf{M} \tilde{\mathbf{x}} + \sum_e k_e \mathbf{G}_e^\top \mathbf{p}_e$$

$$\frac{d\phi}{d\theta} = \mathbf{z}^\top \left( \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta} \mathbf{x}^+ \right)$$

| $\theta$          | $\frac{\partial \mathbf{b}}{\partial \theta}$ | $\frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta}$ | $\frac{d\phi}{d\theta}$                                                     |
|-------------------|-----------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------|
| $\mathbf{x}^-$    | $\frac{1}{h^2} \mathbf{M}$                    | $\mathbf{0}$                                              | $\frac{1}{h^2} \mathbf{z}^\top \mathbf{M}$                                  |
| $\mathbf{v}^-$    | $\frac{1}{h} \mathbf{M}$                      | $\mathbf{0}$                                              | $\frac{1}{h} \mathbf{z}^\top \mathbf{M}$                                    |
| $k_e$             | $\mathbf{G}_e^\top \mathbf{p}_e$              | $\mathbf{G}_e^\top \mathbf{G}_e$                          | $(\mathbf{G}_e \mathbf{z})^\top (\mathbf{p}_e - \mathbf{G}_e \mathbf{x}^+)$ |
| $l_{\text{rest}}$ | $k_e \mathbf{G}_e^\top \hat{\mathbf{e}}$      | $\mathbf{0}$                                              | $k_e (\mathbf{G}_e \mathbf{z})^\top \hat{\mathbf{e}}$                       |

Over a trajectory, a parameter collects one contribution per step, using the adjoint vectors already computed by the backward pass:

$$\frac{d\phi}{d\theta} = \sum_{t=1}^T \mathbf{z}_t^\top \left( \frac{\partial \mathbf{b}_t}{\partial \theta} - \frac{\partial \mathbf{L}_{\text{PD}}}{\partial \theta} \mathbf{x}_t^+ \right)$$

# Projective Dynamics with Dry Frictional Contact

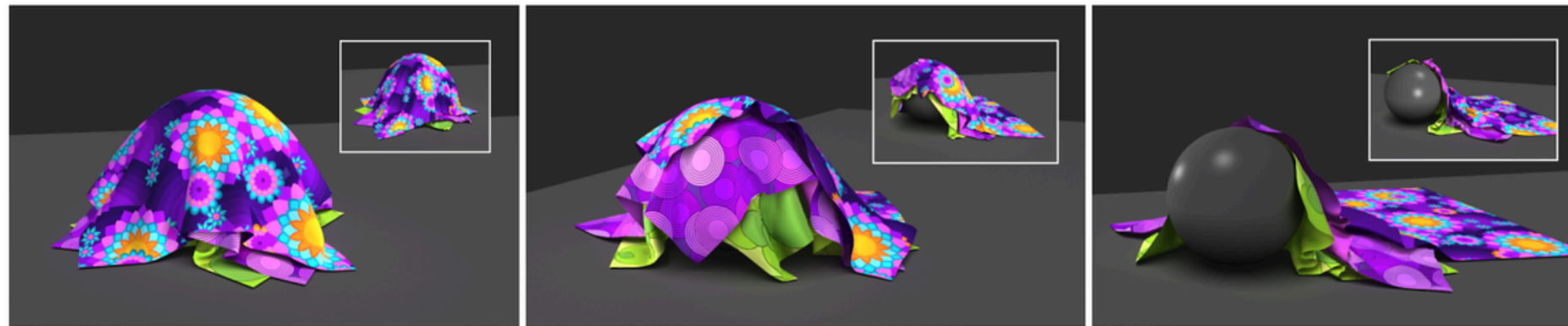
## Projective Dynamics with Dry Frictional Contact

MICKAËL LY, Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LJK, France

JEAN JOUVE, ENS Rennes and Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LJK, France

LAURENCE BOISSIEUX, Univ. Grenoble Alpes, Inria, France

FLORENCE BERTAILS-DESCOUBES, Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LJK, France



[ <https://dl.acm.org/doi/10.1145/3386569.3392396> ]

## | Velocity Based Projective Dynamics

Since Signorini-Coulomb Law for contact with friction are written using velocity, first we derive the Velocity-Based PD variant. We start from the Position-Based PD global step:

$$\mathbf{L}\mathbf{x}^+ = \mathbf{b} \quad \text{with} \quad \begin{aligned} \mathbf{L} &= \mathbf{M} + h^2 \sum k_e \mathbf{G}_e^\top \mathbf{G}_e \\ \mathbf{b} &= \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum k_e \mathbf{G}_e^\top \mathbf{p}_e \end{aligned}$$

We substitute  $\mathbf{x}^+ = (\mathbf{x}^- + h\mathbf{v}^+)$ :

$$\mathbf{L}(\mathbf{x}^- + h\mathbf{v}^+) = \mathbf{b} \quad \Rightarrow \quad \mathbf{L}\mathbf{v}^+ = \underbrace{\frac{1}{h}(\mathbf{b} - \mathbf{L}\mathbf{x}^-)}_{\tilde{\mathbf{b}}}$$

So that we obtain the Velocity-Based global step:

$$\mathbf{L}\mathbf{v}^+ = \tilde{\mathbf{b}}$$

The system matrix is the same as in the Position-Based variant, the only change is in the computation of the right hand side, where we subtract the factor  $\mathbf{L}\mathbf{x}^-$  and divide by  $h$

# | Velocity Based Projective Dynamics

```
function VELOCITYPROJECTIVEDYNAMICS( $\mathbf{x}^-$ ,  $\mathbf{v}^-$ )  
1:    $\tilde{\mathbf{x}} \leftarrow \mathbf{x}^- + h \mathbf{v}^-$   
2:   Initialize  $\mathbf{v}^{(0)}$   
3:    $\mathbf{x}^{(0)} \leftarrow \mathbf{x}^- + h \mathbf{v}^{(0)}$   
4:   for  $k = 0, 1, \dots, N - 1$  do  
5:     for each element  $e$  in parallel do  
6:        $\mathbf{p}_e \leftarrow \text{PROJ}_{\mathcal{M}_e}(\mathbf{G}_e \mathbf{x}^{(k)})$   
7:        $\tilde{\mathbf{b}} \leftarrow \frac{1}{h} \left( \mathbf{M} \tilde{\mathbf{x}} + h^2 \sum_e k_e \mathbf{G}_e^\top \mathbf{p}_e - \mathbf{L} \mathbf{x}^- \right)$   
8:       SOLVE  $\mathbf{L} \mathbf{v}^{(k+1)} = \tilde{\mathbf{b}}$   
9:        $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^- + h \mathbf{v}^{(k+1)}$   
10:   $\mathbf{v}^+ \leftarrow \mathbf{v}^{(N)}$   
11:   $\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$   
end function
```

# PD with Dry Frictional Contact

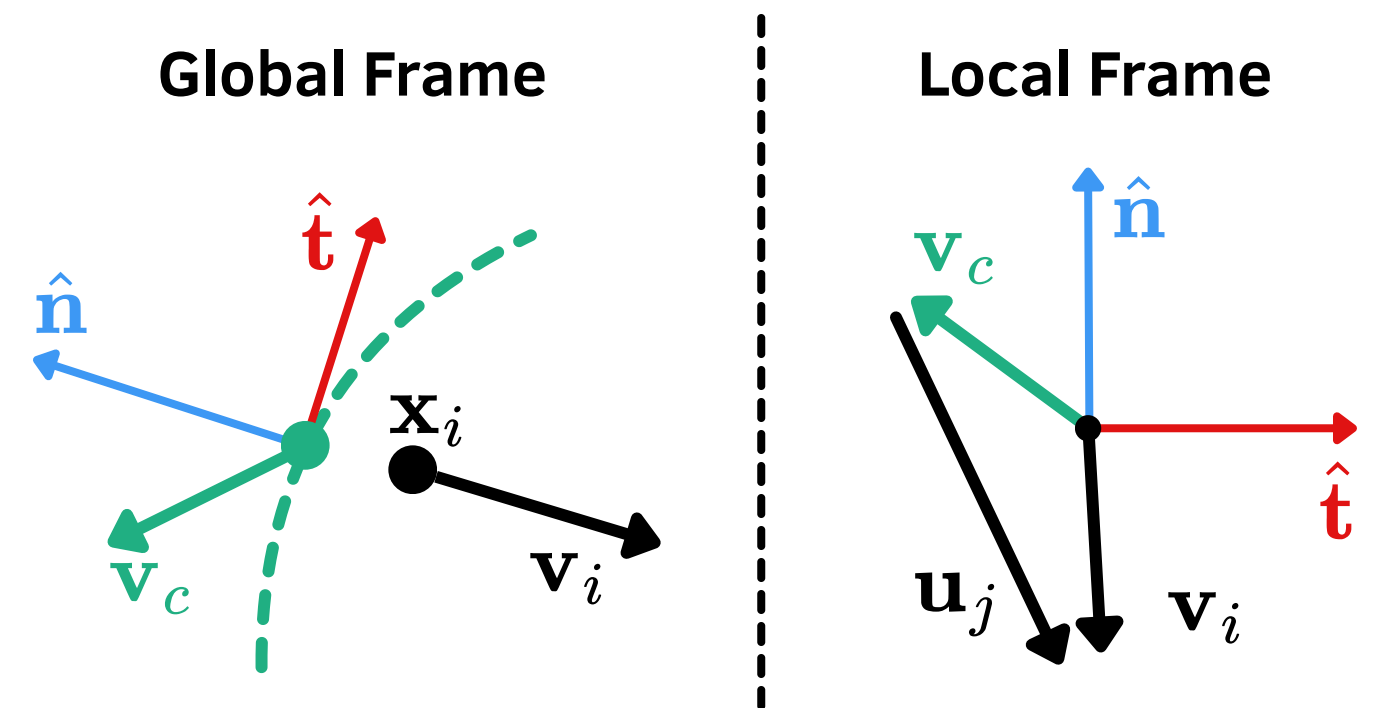
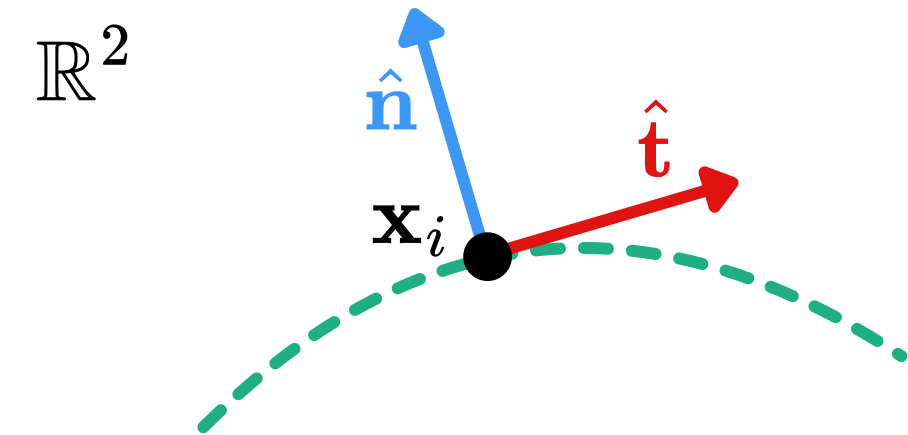
## Contact Frame

$j$  : contact index       $i$  : colliding particle index

$\mathbf{R}_j = [\hat{\mathbf{n}}_j \ \hat{\mathbf{t}}_j \ \hat{\mathbf{t}}'_j]$  : **local frame**, which acts as a rotation matrix from local contact space to world contact.  
 $\mathbf{R}_j^\top$  does the opposite.

$\mathbf{u}_j = \mathbf{R}_j^\top (\mathbf{v}_i - \mathbf{v}_c)$  : **relative velocity**, local frame velocity difference between particle and the obstacle's velocity at the contact point

$\mathbf{r}_j \in \mathbb{R}^3$  : **contact impulse**, applied to the particle by the obstacle, in contact-frame coordinates



# PD with Dry Frictional Contact

## Signorini-Coulomb Law

For each contact the Signorini–Coulomb law constrains the relative velocity and local contact impulse to satisfy:

$$(\mathbf{r}_j, \mathbf{u}_j) \in \mathcal{C}_{\mu_j} \quad \text{with } \mu_j \geq 0 : \text{friction coefficient}$$

Defined by three mutually exclusive and exhaustive cases:

- take-off:  $\mathbf{r}_j = \mathbf{0}, \quad u_{j,N} \geq 0$
- stick:  $\|\mathbf{r}_{j,T}\| \leq \mu_j r_{j,N}, \quad \mathbf{u}_j = \mathbf{0}$
- slip:  $\|\mathbf{r}_{j,T}\| = \mu_j r_{j,N}, \quad u_{j,N} = 0, \quad \mathbf{r}_{j,T} = -\alpha \mathbf{u}_{j,T}, \quad \alpha > 0$

# PD with Dry Frictional Contact

## Signorini-Coulomb Law

For each contact the Signorini–Coulomb law constrains the relative velocity and local contact impulse to satisfy:

$$(\mathbf{r}_j, \mathbf{u}_j) \in \mathcal{C}_{\mu_j} \quad \text{with } \mu_j = 0 : \text{no friction}$$

Defined by two mutually exclusive and exhaustive cases:

- take-off:  $\mathbf{r}_j = \mathbf{0}, \quad u_{j,N} \geq 0$
- contact:  $\mathbf{r}_j = (r_{j,N}, \mathbf{0}), \quad r_{j,N} \geq 0, \quad u_{j,N} = 0, \quad \mathbf{u}_{j,T} \text{ free}$

## | PD with Dry Frictional Contact

Adding contact/frictional impulse to the Velocity-Based PD framework yields:

$$\begin{cases} \mathbf{L} \mathbf{v}^+ = \tilde{\mathbf{b}} + \mathbf{J}_c^\top \mathbf{r}^+ \\ \mathbf{u}^+ = \mathbf{J}_c (\mathbf{v}^+ - \mathbf{v}_c) \\ (\mathbf{r}_j^+, \mathbf{u}_j^+) \in \mathcal{C}_{\mu_j} \quad \forall j = 1, \dots, N_c \end{cases}$$

$\mathbf{J}_c \in \mathbb{R}^{3N_c \times 3n}$  : **contact Jacobian**, a sparse block matrix with one row of blocks per contact, each holding only the transposed local frame at its particle's column:

world  $\rightarrow \mathbf{J}_c \rightarrow$  contact

contact  $\rightarrow \mathbf{J}_c^\top \rightarrow$  world

## | PD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \underbrace{h^2 \sum k_e \mathbf{G}_e^\top \mathbf{G}_e}_{\mathbf{C}} = \mathbf{M} + \mathbf{C}$$

Positive Diagonal

Positive Semi Definite

Using a Jacobi-like split, treating  $\mathbf{M}$  implicitly and lagging  $\mathbf{C}$  to the previous iterate, the global equation becomes:

$$\mathbf{M} \mathbf{v}^{(k+1)} = \underbrace{\tilde{\mathbf{b}} - \mathbf{C} \mathbf{v}^{(k)}}_{\mathbf{f}} + \mathbf{J}_c^\top \mathbf{r}^{(k+1)} = \mathbf{f} + \mathbf{J}_c^\top \mathbf{r}^{(k+1)}$$

The vector  $\mathbf{f}$  is fully computable from previous-iteration data.

Exploiting the diagonal structure of the mass matrix, we write the per-node equation:

$$m_i \mathbf{v}_i^{(k+1)} = \begin{cases} \mathbf{f}_i & \text{if } i \text{ not in contact} \\ \mathbf{f}_i + \mathbf{R}_j \mathbf{r}_j^{(k+1)} & \text{if } i \text{ in contact } j \end{cases}$$

## | PD with Dry Frictional Contact

Using the fact:  $\mathbf{u}_j = \mathbf{R}_j^\top (\mathbf{v}_i - \mathbf{v}_c)$  we obtain:

$$m_i \mathbf{u}_j = \underbrace{\mathbf{R}_j^\top (\mathbf{f}_i - m_i \mathbf{v}_c)}_{\mathbf{d}_j : \text{fully known}} + \mathbf{r}_j \quad (\mathbf{r}_j, \mathbf{u}_j) \in \mathcal{C}_{\mu_j \geq 0} \quad \xrightarrow{\text{with friction}}$$

$\mathbf{d}_j$  is the relative velocity (times mass) the node would have with no contact.  
Choose  $\mathbf{r}_j$  so that  $m_i \mathbf{u}_j = \mathbf{d}_j + \mathbf{r}_j$  satisfies Signorini–Coulomb:

$$\mathbf{r}_j = \begin{cases} \text{take-off:} & \mathbf{0} & \text{if } d_{j,N} \geq 0 \\ \text{stick:} & -\mathbf{d}_j & \text{if } \|\mathbf{d}_{j,T}\| \leq -\mu_j d_{j,N} \\ \text{slip:} & \left( -d_{j,N}, \mu_j d_{j,N} \frac{\mathbf{d}_{j,T}}{\|\mathbf{d}_{j,T}\|} \right) & \text{otherwise} \end{cases}$$

## | PD with Dry Frictional Contact

Using the fact:  $\mathbf{u}_j = \mathbf{R}_j^\top (\mathbf{v}_i - \mathbf{v}_c)$  we obtain:

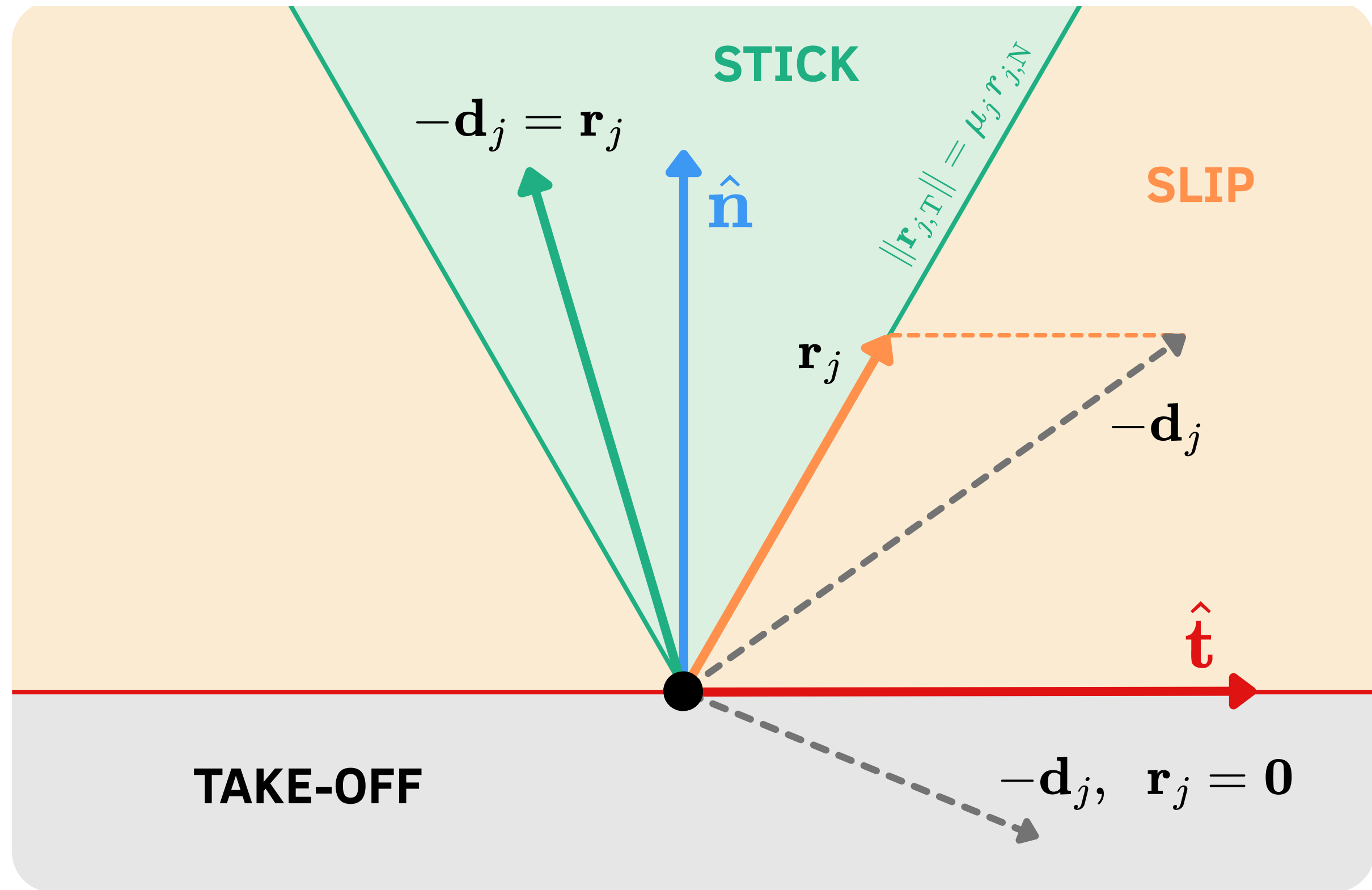
$$m_i \mathbf{u}_j = \underbrace{\mathbf{R}_j^\top (\mathbf{f}_i - m_i \mathbf{v}_c)}_{\mathbf{d}_j : \text{fully known}} + \mathbf{r}_j \quad (\mathbf{r}_j, \mathbf{u}_j) \in \mathcal{C}_{\mu_j=0} \quad \xrightarrow{\text{without friction}}$$

$\mathbf{d}_j$  is the relative velocity (times mass) the node would have with no contact.  
Choose  $\mathbf{r}_j$  so that  $m_i \mathbf{u}_j = \mathbf{d}_j + \mathbf{r}_j$  satisfies Signorini:

$$\mathbf{r}_j = \begin{cases} \text{take-off:} & \mathbf{0} & \text{if } d_{j,N} \geq 0 \\ \text{contact:} & (-d_{j,N}, \mathbf{0}) & \text{otherwise} \end{cases}$$

# PD with Dry Frictional Contact

## Friction Cone



# | PD with Dry Frictional Contact

**function** PDFRICTIONALCONTACTSTEP( $\mathbf{x}^-, \mathbf{v}^-$ )

```
1:  $\tilde{\mathbf{x}} \leftarrow \mathbf{x}^- + h \mathbf{v}^- + h^2 \mathbf{M}^{-1} \mathbf{f}_{\text{ext}}$ 
2:  $\{i, \mathbf{R}_j, \mathbf{v}_c\}_j \leftarrow \text{DETECTCONTACTS}(\tilde{\mathbf{x}})$ 
3:  $\mathbf{x}^{(0)} \leftarrow \tilde{\mathbf{x}}, \quad \mathbf{v}^{(0)} \leftarrow \mathbf{v}^-$ 
4: for  $k = 0, 1, \dots, N - 1$  do
5:   for each element  $e$  in parallel do
6:      $\mathbf{p}_e \leftarrow \text{PROJ}_{\mathcal{M}_e}(\mathbf{G}_e \mathbf{x}^{(k)})$ 
7:      $\tilde{\mathbf{b}} \leftarrow \frac{1}{h} \left( \mathbf{M} \tilde{\mathbf{x}} + h^2 \sum_e k_e \mathbf{G}_e^\top \mathbf{p}_e - \mathbf{L} \mathbf{x}^- \right)$ 
8:      $\mathbf{f} \leftarrow \tilde{\mathbf{b}} - \mathbf{C} \mathbf{v}^{(k)}$ 
9:     for each contact  $j$  in parallel do
10:       $\mathbf{d}_j \leftarrow \mathbf{R}_j^\top (\mathbf{f}_i - m_i \mathbf{v}_c)$ 
11:       $\mathbf{r}_j \leftarrow \text{UPDATECONTACTFORCE}(\mathbf{d}_j, \mu_j)$ 
12:       $\tilde{\mathbf{b}}_i += \mathbf{R}_j \mathbf{r}_j$ 
13:   SOLVE  $\mathbf{L} \mathbf{v}^{(k+1)} = \tilde{\mathbf{b}}$ 
14:    $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^- + h \mathbf{v}^{(k+1)}$ 
15:  $\mathbf{v}^+ \leftarrow \mathbf{v}^{(N)}$ 
16:  $\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$ 
end function
```

## Collision Detection

Find the particles that will hit an obstacle, using the predicted positions. Each contact gets its frame and the obstacle velocity

## Local step: Constraints

Project each element onto its rest shape, as in standard PD, then build the right-hand side from these projections.

## Local Step: Collisions

Estimate each contacting particle's momentum without contact. Choose the impulse that satisfies Signorini–Coulomb, then add it to the right-hand side

## Global Step

One solve with the prefactored System Matrix. Contact changes only the right-hand side.

# Differentiable PD with Dry Frictional Contact

DiffCloth: Differentiable Cloth Simulation with Dry Frictional Contact

YIFEI LI and TAO DU, MIT CSAIL, USA

KUI WU, Tencent Lightspeed & Quantum Studios, USA

JIE XU and WOJCIECH MATUSIK, MIT CSAIL, USA



<https://people.csail.mit.edu/liyifei/publication/diffcloth/>

# DiffPD with Dry Frictional Contact

## Position-PD

$$\mathbf{L} = \frac{1}{h^2} \mathbf{M} + \sum k_e \mathbf{G}_e^\top \mathbf{G}_e$$

$$\mathbf{b} = \frac{1}{h^2} \mathbf{M} \tilde{\mathbf{x}} + \sum k_e \mathbf{G}_e^\top \mathbf{p}_e$$

At convergence it holds:

$$\mathbf{L}(\theta) \mathbf{x}^+ = \mathbf{b}(\theta, \mathbf{x}^+)$$

Steps to compute Gradient:

- Find **Real Hessian**:

$$\mathbf{H}_{\text{PD}} = \mathbf{L} - \Delta \mathbf{L}$$

- Find method to compute **Adjoint Vector**:

$$\mathbf{L} \mathbf{z}^{(k+1)} = \Delta \mathbf{L} \mathbf{z}^{(k)} + \left( \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top$$

- Find formula to compute **Gradient**:

$$\frac{d\phi}{d\theta} = \mathbf{z}^\top \left( \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}}{\partial \theta} \mathbf{x}^+ \right)$$

## Velocity-PD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C} \quad \mathbf{C} = h^2 \sum k_e \mathbf{G}_e^\top \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M} \tilde{\mathbf{x}} + h^2 \sum k_e \mathbf{G}_e^\top \mathbf{p}_e \quad \tilde{\mathbf{b}} = \frac{1}{h} (\mathbf{b} - \mathbf{L} \mathbf{x}^-)$$

$$\mathbf{r} = \Phi(\mathbf{f}) \quad \mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C} \mathbf{v}^+$$

At convergence it holds:

$$\mathbf{L}(\theta) \mathbf{v}^+ = \tilde{\mathbf{b}}(\theta, \mathbf{v}^+) + \mathbf{r}(\theta, \mathbf{v}^+)$$

Steps to compute Gradient:

- Find **Real Hessian**
- Find method to compute **Adjoint Vector**
- Find formula to compute **Gradient**

# | DiffPD with Dry Frictional Contact

$$\begin{aligned}
 \mathbf{L} &= \mathbf{M} + \mathbf{C} & \mathbf{C} &= h^2 \sum k_e \mathbf{G}_e^\top \mathbf{G}_e & \Delta \mathbf{L} &= \sum k_e \mathbf{G}_e^\top \mathbf{\Gamma}_e \mathbf{G}_e \\
 \mathbf{b} &= \mathbf{M} \tilde{\mathbf{x}} + h^2 \sum k_e \mathbf{G}_e^\top \mathbf{p}_e & \tilde{\mathbf{b}} &= \frac{1}{h} (\mathbf{b} - \mathbf{L} \mathbf{x}^-) & \mathbf{\Gamma}_e &= \frac{\partial \mathbf{p}_e}{\partial (\mathbf{G}_e \mathbf{x})} \\
 \mathbf{r} &= \Phi(\mathbf{f}) & \mathbf{f} &= \tilde{\mathbf{b}} - \mathbf{C} \mathbf{v}^+ & \mathbf{\Gamma}_c &= \frac{\partial \Phi}{\partial \mathbf{f}}
 \end{aligned}$$

- Find **Real Hessian**:

$$\mathbf{H}_c = \overbrace{\mathbf{M} + h^2 \sum k_e \mathbf{G}_e^\top \mathbf{G}_e}^{\mathbf{L}} - \overbrace{h^2 \sum k_e \mathbf{G}_e^\top \mathbf{\Gamma}_e \mathbf{G}_e}^{\Delta \mathbf{L}_c} - \overbrace{\mathbf{\Gamma}_c (h^2 \sum k_e \mathbf{G}_e^\top \mathbf{\Gamma}_e \mathbf{G}_e - h^2 \sum k_e \mathbf{G}_e^\top \mathbf{G}_e)}^{\Delta \mathbf{R}^\top}$$

$$\mathbf{H}_c = \mathbf{L} - \Delta \mathbf{L}_c - \Delta \mathbf{R}^\top \quad \text{with} \quad \begin{aligned} \Delta \mathbf{L}_c &= h^2 \Delta \mathbf{L} \\ \Delta \mathbf{R} &= (\Delta \mathbf{L}_c - \mathbf{C}) \mathbf{\Gamma}_c^\top \end{aligned}$$

- Find method to compute **Adjoint Vector**:

**Iterative Solver**

$$(\mathbf{L} - \Delta \mathbf{L}_c - \Delta \mathbf{R}) \mathbf{z} = \left( \frac{\partial \phi}{\partial \mathbf{v}^+} \right)^\top \quad \Rightarrow \quad \mathbf{L} \mathbf{z}^{(k+1)} = (\Delta \mathbf{L}_c + \Delta \mathbf{R}) \mathbf{z}^{(k)} + \left( \frac{\partial \phi}{\partial \mathbf{v}^+} \right)^\top$$

# | DiffPD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C}$$

$$\mathbf{C} = h^2 \sum k_e \mathbf{G}_e^\top \mathbf{G}_e$$

$$\Delta \mathbf{L} = \sum k_e \mathbf{G}_e^\top \mathbf{\Gamma}_e \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M} \tilde{\mathbf{x}} + h^2 \sum k_e \mathbf{G}_e^\top \mathbf{p}_e$$

$$\tilde{\mathbf{b}} = \frac{1}{h} (\mathbf{b} - \mathbf{L} \mathbf{x}^-)$$

$$\mathbf{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial (\mathbf{G}_e \mathbf{x})}$$

$$\mathbf{r} = \Phi(\mathbf{f})$$

$$\mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C} \mathbf{v}^+$$

$$\mathbf{\Gamma}_c = \frac{\partial \Phi}{\partial \mathbf{f}}$$

Adjoint Vector Iterative Solver:

$$\mathbf{L} \mathbf{z}^{(k+1)} = h^2 \Delta \mathbf{L} \bar{\mathbf{z}}^{(k)} + \mathbf{C} (\mathbf{z}^{(k)} - \bar{\mathbf{z}}^{(k)}) + \left( \frac{\partial \phi}{\partial \mathbf{v}^+} + h \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top \quad \bar{\mathbf{z}} = (\mathbf{I} + \mathbf{\Gamma}_c)^\top \mathbf{z}$$

# | DiffPD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C}$$

$$\mathbf{C} = h^2 \sum k_e \mathbf{G}_e^\top \mathbf{G}_e$$

$$\Delta \mathbf{L} = \sum k_e \mathbf{G}_e^\top \mathbf{\Gamma}_e \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M} \tilde{\mathbf{x}} + h^2 \sum k_e \mathbf{G}_e^\top \mathbf{p}_e$$

$$\tilde{\mathbf{b}} = \frac{1}{h} (\mathbf{b} - \mathbf{L} \mathbf{x}^-)$$

$$\mathbf{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial (\mathbf{G}_e \mathbf{x})}$$

$$\mathbf{r} = \Phi(\mathbf{f})$$

$$\mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C} \mathbf{v}^+$$

$$\mathbf{\Gamma}_c = \frac{\partial \Phi}{\partial \mathbf{f}}$$

## Adjoint Vector Iterative Solver:

$$\mathbf{L} \mathbf{z}^{(k+1)} = \underbrace{h^2 \Delta \mathbf{L} \bar{\mathbf{z}}^{(k)}}_{\text{Local Elastic}} + \underbrace{\mathbf{C} (\mathbf{z}^{(k)} - \bar{\mathbf{z}}^{(k)})}_{\text{Local Contact}} + \left( \frac{\partial \phi}{\partial \mathbf{v}^+} + h \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top$$

The constant system matrix of the forward PD, already factorized: each iteration costs one back-substitution.

The loss usually depends on positions. Since  $\mathbf{x}^+ = \mathbf{x}^- + h \mathbf{v}^+$ , the position gradient reaches velocity scaled by h.

$$\bar{\mathbf{z}} = (\mathbf{I} + \mathbf{\Gamma}_c)^\top \mathbf{z}$$

**Filtered adjoint:** at contact nodes it removes the components the contact controls.

It is **0** for nodes not in contact.

Same local/global structure as DiffPD: two parallel local steps and one global solve. Without contact,  $\bar{\mathbf{z}} = \mathbf{z}$ , and we recover the DiffPD iteration.

# DiffPD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C}$$

$$\mathbf{C} = h^2 \sum k_e \mathbf{G}_e^\top \mathbf{G}_e$$

$$\Delta \mathbf{L} = \sum k_e \mathbf{G}_e^\top \mathbf{\Gamma}_e \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M} \tilde{\mathbf{x}} + h^2 \sum k_e \mathbf{G}_e^\top \mathbf{p}_e$$

$$\tilde{\mathbf{b}} = \frac{1}{h} (\mathbf{b} - \mathbf{L} \mathbf{x}^-)$$

$$\mathbf{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial (\mathbf{G}_e \mathbf{x})}$$

$$\mathbf{r} = \Phi(\mathbf{f})$$

$$\mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C} \mathbf{v}^+$$

$$\mathbf{\Gamma}_c = \frac{\partial \Phi}{\partial \mathbf{f}}$$

## Adjoint Vector Iterative Solver:

$$\mathbf{L} \mathbf{z}^{(k+1)} = h^2 \Delta \mathbf{L} \bar{\mathbf{z}}^{(k)} + \mathbf{C} (\mathbf{z}^{(k)} - \bar{\mathbf{z}}^{(k)}) + \left( \frac{\partial \phi}{\partial \mathbf{v}^+} + h \frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top \quad \bar{\mathbf{z}} = (\mathbf{I} + \mathbf{\Gamma}_c)^\top \mathbf{z}$$

## Formula for General Parameter Gradient:

### Specialization:

Gradient of previous  
step Velocity:

$$\frac{d\phi}{d\mathbf{v}^-} = \bar{\mathbf{z}}^\top \mathbf{M}$$

Gradient of previous  
step Position:

$$\frac{d\phi}{d\mathbf{x}^-} = \bar{\mathbf{z}}^\top \left( h \Delta \mathbf{L} - \frac{1}{h} \mathbf{C} \right) + \frac{\partial \phi}{\partial \mathbf{x}^+}$$

Gradient of shared stiffness of  
all springs along all trajectory:

$$\frac{d\phi}{dk} = h \sum_{t=1}^T \sum_e (\bar{\mathbf{z}}_{t,i} - \bar{\mathbf{z}}_{t,j})^\top (\mathbf{p}_e - \mathbf{e})_t$$

$$\frac{d\phi}{d\theta} = \bar{\mathbf{z}}^\top \left( \frac{\partial \tilde{\mathbf{b}}}{\partial \theta} - \frac{\partial \mathbf{L}}{\partial \theta} \mathbf{v}^+ \right)$$